

Non-interactive Blind Signatures with Threshold Issuance

Foteini Baldimtsi
George Mason University
& Mysten Labs
foteini@gmu.edu

Lucjan Hanzlik
CISPA Helmholtz Center
for Information Security
hanzlik@cispa.de

Aayush Yadav
George Mason University
ayadav5@gmu.edu

Abstract

Non-interactive blind signatures (NIBS) capture the minimal setting of blind signatures where the message space is restricted to unstructured random strings. They enable a signer to pre-compute presignatures without prior interaction, while ensuring that only the intended recipient can derive the corresponding blind signature.

In this work, we consider the problem of threshold issuance of NIBS. Specifically, we introduce the notion of non-interactive threshold blind signatures (NITBS), where a user obtains partial presignatures from a threshold of signers and locally combines them into a valid blind signature. We provide a formal treatment of this primitive by defining the corresponding security notions of blindness and one-more unforgeability. We then present the first concrete construction of NITBS, obtained by adapting the Pointcheval-Sanders (PS) signature scheme, and establish its security in the algebraic group model. Our micro-benchmarking results show that our construction attains the smallest presignature and signature sizes and the fastest issuance among all existing NIBS schemes.

Contents

1	Introduction	3
1.1	Technical overview	4
1.2	Related work	8
2	Preliminaries	9
2.1	Signature schemes	9
2.2	Non-interactive zero knowledge proof systems	10
2.3	Non-interactive blind signature schemes	11
2.4	Threshold secret sharing	12
2.5	Bilinear groups	13
2.6	Hardness assumptions	13
3	Non-interactive Threshold Blind Signatures	15
4	NIBS from PS Signatures	18
4.1	Security	20
4.2	The concrete NIZK proof system	26
5	NITBS from PS Signatures	29
5.1	Security	31
5.2	Efficiency	34
6	Extensions	35
6.1	Tagged NI(T)BS	35
6.2	NITBS with verifiable presignatures	35
6.3	NI(T)BS with batch issuance	36
6.3.1	Concrete NIZK for NI(T)BS with batch issuance	37

1 Introduction

Blind signatures are a core primitive in several privacy preserving applications. They were introduced by Chaum [Cha83] as a way to instantiate a private e-cash protocol and, since then, have found further applications in e-voting [CGT06], anonymous credentials [PZ13, BL13, FHS15, DGS⁺18, KRB⁺24], direct anonymous attestation [BCC04], and cryptocurrency mixers [HBG16, HAB⁺17]. The past few years have seen a resurgence of interest in blind signatures, fueled by active standardization initiatives [DIW24, PVW24, CDVW24] and broad industry deployment [App21, Goo24, MRFH24].

Typically, a blind signature is presented as a two-party protocol between a signer with secret signing key sk , and a recipient with a message m and the signer’s public verification key pk . The recipient blinds m and sends the blinded value to the signer, who in turn returns a (pre)signature $\bar{\sigma}$ on the blinded message. The recipient then “un-blinds” $\bar{\sigma}$ to obtain (m, σ) verifiable under pk . In addition to the classical notion of unforgeability, a blind signature should also guarantee blindness which states that the resulting pair (m, σ) is *unlinkable* to the issuance transcript.

As defined above, blind signatures require at least one round (i.e., two moves) of interaction between the signer and the recipient [Cha83, PS96, PS00b]. Even so, interaction introduces latency and operational overhead in high-throughput systems. In large-scale settings where many credentials must be issued quickly and reliably, even a single interactive round can act as a scalability bottleneck.

More importantly, in the various privacy-sensitive applications where blind signatures are commonly used, such as digital cash, ticketing, and electronic voting, centralized issuance creates a single point of failure. If a signer is compromised, unauthorized signatures can be issued, breaking the integrity of the system. If it suffers outages or denial-of-service, issuance halts and recipients are blocked. These weaknesses stem directly from the concentration of risk on a single issuing authority.

Non-interactive blind signatures (NIBS). Recent works [Han23, BCGY24, HPZ25, BHNY25] have shown that interaction is not inherently necessary in order to issue blind signatures. In certain settings, where the message to be signed is random, meaning that it is not chosen by the recipient or drawn from a structured distribution, blind signature issuance can be made *non-interactive*. This condition arises naturally in applications where the signed message is essentially a context-specific random identifier (such as a nonce) rather than a recipient-selected string. Looking back at e-cash, for example, the signed message which serves as the coin identifier is simply a uniformly random string. Similarly, in vouchers or admission tickets, the message can be an unpredictable value generated to uniquely label a token that will later be presented for redemption.

A key practical advantage of NIBS is that an issuer can *asynchronously* create “presignatures” that can later be converted to full signatures by the corresponding recipient. Thus the signer can *pre-compute* presignatures offline, thereby decoupling expensive cryptographic operations from peak demand. This removes the need to process large volumes of concurrent online signing requests, substantially reducing latency and operational bottlenecks described above.

Threshold blind signatures. A prominent solution to address the risks introduced by single points of failure is to leverage *threshold* issuance, i.e., distributing the signer’s operation across multiple parties. Recall that in a threshold blind signature scheme, the issuer’s secret key is secret-shared among n parties (e.g., via distributed key-generation), and the recipient can interact with any t of them in order to eventually obtain a valid signature. Unlike standard threshold signatures, the signers must learn nothing about the message being signed, and the resulting signature must remain unlinkable to the issuance transcript. Despite their obvious appeal relatively few constructions are known [VZK03, KM15, CKM⁺23]; this likely reflects the difficulty of simultaneously (i) binding partial signatures to a common signing instance, (ii)

providing robustness against misbehaving parties, and (iii) preserving message privacy and unlinkability in the presence of concurrent sessions.

The open problem. While NIBS address the overhead of interaction and threshold blind signatures address the risk of compromise, no existing work achieves both properties at once. A *non-interactive threshold blind signature* would allow any t issuers to contribute presignature shares that aggregate non-interactively into a full signature, while still preserving blindness and unforgeability. Designing such a primitive is non-trivial, yet highly desirable as it combines the efficiency of non-interactivity with the robustness and trust distribution of threshold cryptography. In this work, we show this is achievable by constructing the first threshold NIBS with non-interactive (presignature) aggregation.

Our contributions. We summarize our contributions as follows:

- We design a formal framework for Non-interactive Threshold Blind Signatures (NITBS) in Section 3. Our definitions delicately capture the combined guarantees of one-more unforgeability and blindness in the non-interactive and threshold setting. We also characterize two hierarchical levels of one-more unforgeability achievable in the NITBS setting.
- As an intermediate step to our threshold NIBS construction, in Section 4 we design a practical NIBS construction from Pointcheval-Sanders (PS) signatures. This is an standalone contribution since, compared to prior NIBS proposals, our construction has important efficiency gains. In particular, our construction attains the *smallest-known presignature and signature size* and improves issuance efficiency, which is especially advantageous when presignatures are pre-computed or stored (such as in Privacy Pass).
- We present the first full NITBS scheme that supports non-interactive aggregation of presignatures in Section 5. We prove security in pairing-friendly groups: blindness reduces to the hardness of the q -DDHI problem, and one-more unforgeability reduces to a threshold-variant of the PS assumption that, we show, reduces to the hardness of the one-more co-Discrete Logarithm problem (in the algebraic group model). Our scheme tolerates adaptive corruptions of up to $(t - 1)$ issuers. We estimate that for the BLS12-381 curve, our scheme requires 0.6 ms issuance time, and less than 5 ms to obtain the final blind signature for thresholds of up to 5 (with growth roughly linear in t), and 4 ms for verification. On the same curve, the receiver’s public key is around 112 bytes, the presignature size is 96 bytes and the final signature is around 192 bytes.
- Finally, in Section 6 we present a series of extensions including (i) *tagged* NI(T)BS, which permits inclusion of non-blinded auxiliary data (analogously to partially blind signatures); (ii) NITBS with *verifiable presignatures*, which allows recipients to check the presignature shares before aggregation; and (iii) *batched* NI(T)BS, which enables a bounded number of final signatures to be derived from a single valid presignature.

1.1 Technical overview

Before jumping into the technical nuances of our approach, let us begin by recalling some necessary background details.

Recalling non-interactive blind signatures. In a non-interactive blind signature scheme, a signer with key-pair (pk, sk) , issues a *presignature* $\bar{\sigma}$, under a recipient public key pk_R and some public *context* c ¹,

¹This can be thought of as a nonce computed using some known public information such as $H(pk_R \parallel \text{timestamp})$.

given which the recipient can derive a message m , and a (blind) signature σ such that (m, σ) verifies under pk . Notably, the message m is an unstructured (pseudo)random value derived by the recipient as a combination of its secret key sk_R and the context c , over which it possesses the presignature.

In terms of security requirements, a NIBS scheme must be *unforgeable*, intuitively meaning that a recipient should not be able to derive a valid (m, σ) pair on its own; and it must be *blind*, meaning that a signer should not be able to link a (m, σ) pair to any specific issuance, and by extension to any specific recipient. Strictly speaking, these correspond to the notions of *one-more* unforgeability, and *recipient* and *context* blindness. We remark that while achieving concurrent security can be a challenge in multi-round blind signature protocols, thanks to their non-interactivity, NIBS are concurrently secure by default.

One-more unforgeability in the NIBS context is a natural extension of the same property for traditional blind signatures. As defined in [Han23], the adversary is granted access to a presignature oracle, and after obtaining ℓ presignatures for (pk_R, c) pairs of its choice, it must produce $(\ell + 1)$ valid signatures to succeed.

Blindness is captured by two distinct security experiments [Han23, BCGY24]: (i) recipient blindness, ensuring that a malicious signer cannot determine which of two possible presignatures led to the final signature; and (ii) context blindness, which ensures that when a signer issues two presignatures to a specific recipient, they cannot link the final message-signature pairs to the respective presignatures. Both definitions are given as indistinguishability-based experiments.

Concretely, in the recipient blindness experiment, the adversary is given two recipient public keys (pk_{R_0}, pk_{R_1}) along with an oracle for each public key, and outputs two presignatures $(\bar{\sigma}_0, \bar{\sigma}_1)$ and contexts (c_0, c_1) , one for each key. The challenger then computes the final signature-message pairs $(m_0, \sigma_0), (m_1, \sigma_1)$ using the presignatures. The scheme satisfies recipient blindness if the attacker cannot link the final signature-message pairs, presented in random order, to the corresponding issuance. The experiment for context blindness is similar, but the adversary is given only a single recipient public key pk_R and correspondingly, an oracle for pk_R , and outputs two presignatures $(\bar{\sigma}_0, \bar{\sigma}_1)$ and context (c_0, c_1) . The challenger then computes the final signature-message pairs $(m_0, \sigma_0), (m_1, \sigma_1)$ from these presignatures. As before, the scheme is said to satisfy context blindness if the attacker cannot link the final signature-message pairs to their issuance.

A new framework for thresholdizing NIBS. Syntactically, NIBS can be extended to admit threshold signing in a natural way. Informally, a t -out-of- n non-interactive threshold blind signature scheme admits n signers, with distributed secret keys sk_i , and a common public key mpk such that, given *partial* presignatures $\bar{\sigma}_i$ from at least t signers, under a common public context c , a recipient with public key pk_R can use the corresponding sk_R to derive a message m , and a (blind) signature σ such that (m, σ) verifies under mpk .

In order for an NITBS scheme to be non-trivial, we require that it must be *efficient* in the sense that the final message and signature sizes should be *independent* of n . Further, an NITBS scheme must also extend the NIBS properties of one-more unforgeability, and recipient and context blindness to the threshold setting. As a matter of fact, the blindness properties can be formalized almost identically to their NIBS counterparts since the point-of-view of the signer(s) is effectively the same in either case. However, this is not the case for unforgeability as the threshold and non-threshold views clearly differ from the recipient's point-of-view. For our NITBS unforgeability definition, we adapt the corresponding notions from the recent work of Lehmann et al. [LNÖ25] for interactive threshold blind signatures. In particular, after stripping away the session-related bookkeeping details required for the interactive setting, we are left with two notions of one-more unforgeability of NITBS against a malicious recipient who can adaptively corrupt up to $(t - 1)$ signers and learn their secret signing keys. The distinction lies in how presignature

queries to honest signers are handled:

- (i) In the first case, the adversary may query individual honest signers, obtaining a presignature share from the i^{th} signer in response to a query (i, pk_R, c) . This defines a stronger notion, as the adversary has more control over the query count ℓ , which increments only when the number of presignature shares for a given (pk_R, c) pair exceeds $(t - 1)$ (including via corruptions).
- (ii) In the second case, each presignature query (pk_R, c) yields presignature shares from *all* of them. This defines a weaker notion, as every query automatically provides enough presignature shares to derive the final signature (including those obtained via corruption), and the query count ℓ increments with each such query.

These two notions correspond, roughly, to the OM-Unf-1 and OM-Unf-0 security notions of interactive threshold blind signatures.

NIBS from PS Signatures. We will now approach the problem of designing NITBS in a piece-wise manner. We begin by showing how to build a NIBS scheme from the signature scheme due to Pointcheval and Sanders [PS16], and then attempt to apply standard threshold secret sharing techniques over the signers' secrets to obtain NITBS².

To recall the PS signature scheme, let $g \in \mathbb{G}$ and $\tilde{g} \in \tilde{\mathbb{G}}$ be the generators of two pairing-friendly groups, then in the PS signature scheme, the signature on a message vector $(\mu_1, \dots, \mu_k) \in \mathbb{F}^k$ is given by $\Psi := (\psi_1 = h, \psi_2 = h^{x + \sum_{i=1}^k y_i \mu_i})$ where $(x, y_1, y_2, \dots, y_n) \in \mathbb{F}^{k+1}$ is the secret signing key sk , and given the corresponding public verification key is $\text{pk} := (\tilde{X} = \tilde{g}^x, \tilde{Y}_1 = \tilde{g}^{y_1}, \tilde{Y}_2 = \tilde{g}^{y_2}, \dots, \tilde{Y}_k = \tilde{g}^{y_k}) \in \tilde{\mathbb{G}}^{k+1}$, the signature can be verified by the pairing equation $e(\psi_2, \tilde{g}) \stackrel{?}{=} e(\psi_1, \tilde{X} \cdot \prod_{i=1}^k \tilde{Y}_i^{\mu_i})$. The security of the PS signature scheme is based on the PS problem, which essentially requires that, given $g, \tilde{g}, \tilde{g}^x, \tilde{g}^y$ and an oracle that, on input $\mu \in \mathbb{F}$, outputs $(h, h^{x+y\mu})$ for h chosen uniformly over $\mathbb{G}$, it is hard to find $(\mu^*, h^*, (h^*)^{x+y\mu^*})$ for any fresh $\mu^* \in \mathbb{F}$ and any non-trivial h^* .

Now, in order to construct a NIBS scheme using this general template, the obvious approach would be to have the signer with key-pair $\text{pk} := (\tilde{X}, \tilde{Y}_1, \tilde{Y}_2), \text{sk} := (x, y_1, y_2)$ sign (c, pk_R) and send it as the presignature $\bar{\sigma}$, and then have the recipient re-randomize $\bar{\sigma}$. However, pk_R will generally be a group element, and therefore cannot be signed directly as a message. Fortunately, this is easily resolved. In particular, let $\text{sk}_R := \alpha$, and define $\text{pk}_R := g^\alpha$, then the signer can instead compute values $h = g^r$ and $H = (g^{x+y_1c} \cdot \text{pk}_R^{y_2})^r$ for $r \leftarrow \mathbb{F}$. Since $H = (g^r)^{x+y_1c+y_2\alpha}$, clearly $\bar{\sigma} := (h, H)$ is a valid PS signature on (c, sk_R) . Of course, given $\bar{\sigma}$, the recipient can re-randomize it to obtain a fresh signature Ψ that is unlinkable to $\bar{\sigma}$, but it still cannot send the message in the clear. Instead, it must compute $m := g^{1/(c+\alpha)}$ and then prove using a non-interactive zero-knowledge (NIZK) proof that it knows c and α such that Ψ is a valid PS signature under pk over the message (c, α) and $m = g^{1/(c+\alpha)}$. The resulting proof π (along with the re-randomized PS signature Ψ) is then the final blind signature on message m . To maximize efficiency of our construction, we design a Σ -protocol based on the Schnorr identification scheme [Sch90].

Intuitively, blindness follows from the zero-knowledge of π , indistinguishability of Ψ from uniform (under the Strong Decisional Diffie-Hellman problem [PS00a]) and pseudorandomness of m (under the hardness of Decisional q -Diffie Hellman Inverse problem [DY05]). Similarly, unforgeability can be shown by reducing to the hardness of the PS problem modulo two small caveats. First, we actually cannot

²We choose PS signatures specifically due to their efficiency and, more importantly, their amenability to this thresholdizing template. This choice, however, does not preclude other signature schemes such as BBS [BBS04] and BBS+ [CDL16, ASM06] which have similar profiles; we leave that as future research.

directly sign on c . Rather, at a very high level, we must replace it with a programmable value which will come from the PS challenger. Thus, the signer actually signs the message $(\kappa := H(\text{pk}_R, c), \text{sk}_R)$ where we model H as a random oracle whose outputs are tied to the PS oracle. Secondly, we actually require the NIZK proof to satisfy the stronger property of *online extractability* since a naive forking approach would incur soundness loss exponential in ℓ , the number of presignature queries the adversary makes. To obtain online extractability without sacrificing the efficiency of the Schnorr protocol [Sch90], we work in the algebraic group model (AGM) [FKL18].

Remark 1.1. This scheme offers faster issuance, and smaller presignatures and final signatures than the previous state-of-the-art NIBS scheme due to Hanzlik [Han23]. Concretely, issuance requires 3 exponentiations in $\mathbb{G}$, 1 multiplication in $\mathbb{G}$, and 1 H computation into $\mathbb{F}$, compared to 5 exponentiations in $\mathbb{G}$ and 2 multiplications in $\mathbb{G}$ in [Han23]. The presignature size here is just 2 elements in $\mathbb{G}$, which is roughly half the presignature size in [Han23], where it additionally includes 1 element in $\tilde{\mathbb{G}}$. The same is true for the final signature.

Constructing threshold NIBS from PS signatures. Given the NIBS scheme from PS signatures, we will attempt to apply the standard thresholdizing trick for some threshold t by simply distributing t -out-of- n linear secret shares of $\text{sk} := (x, y_1, y_2)$ among n signers³. Let $[x]_i$ denote the i^{th} share of x (similarly $[y_1]_i$ and $[y_2]_i$). Then, suppose that each signer, as before, issues $\bar{\sigma}_i := \left(h_i = g^{r_i}, H_i = g^{[x]_i + [y_1]_i \kappa} \cdot \text{pk}_R^{[y_2]_i} \right)$ over some shared context c ⁴. Could we then have the recipient simply prove that it obtained at least t valid presignature shares? Unfortunately, if we wish to keep the efficiency gained by using Schnorr proofs, this is not an option as this would result in a proof size that grows with $t = O(n)$. So we must find a way for the recipient to locally ‘aggregate’ the $\geq t$ presignature shares into a single valid PS signature.

The first hurdle in accomplishing this appears in the base h_i of the PS signature. In other words, since each signer signs over a distinct base h_i , there is no clear way to combine the H_i ’s. To address this we will instead have all signers compute a common base h that depends exclusively on the pk_R and c being signed. Simply put, the signers compute $h := H_{\mathbb{G}}(\text{pk}_R, c)$ (for clarity, we will now write $\kappa := H_{\mathbb{Z}}(\text{pk}_R, c)$) but note the second hurdle, that H_i ’s can no longer be computed as before since the signer does not know h^α . Instead, the i^{th} signer factors it into two parts as $A_i := h^{[x]_i + [y_1]_i \kappa} \cdot g$ and $B_i := (\text{pk}_R)^{-1} \cdot h^{[y_2]_i}$ and sends the two values over as $\bar{\sigma}_i$ to the recipient, who can compute the corresponding PS signature $(h, A_i \cdot B_i^\alpha = h^{[x]_i + [y_1]_i \kappa + [y_2]_i \alpha})$ with base h . A minor subtlety here is that the recipient’s public key pk_R must now be $g^{1/\alpha}$, so that the PS signature is still over α . Another technical hurdle that is not apparent from this sketch, but becomes apparent when describing the proof is that the signers will need to introduce some additional randomness into their presignatures for the unforgeability of the scheme. That is, they must in fact compute $A_i := h^{[x]_i + [y_1]_i \kappa} \cdot g^{r_i}$ and $B_i := (\text{pk}_R)^{-r_i} \cdot h^{[y_2]_i}$.

One can now easily see that any recipient with at least t such PS signatures can use the share recombination algorithm (in the exponent) of the linear threshold secret sharing scheme to obtain the final PS signature $(h, h^{x + y_1 \kappa + y_2 \alpha})$. The rest of the protocol proceeds as before, with the recipient proving knowledge of κ and α such that the (re-randomized) PS signature Ψ is valid under pk over the message (κ, α)

³We remark that we arrived at our NIBS construction by explicitly engineering the issuance computation so that it decomposes into additive shares that independent signers can produce non-interactively while still yielding a valid blind signature upon aggregation. This property is not present in prior NIBS schemes and the general secret-sharing recipe does not apply directly to existing designs [Han23, BCGY24, BGY25, HPZ25].

⁴If the scheme allowed for arbitrary combination of presignature shares on different contexts, a recipient could mix-and-match presignatures. The simplest way to avoid this is to keep the context fixed for each signature. As an example: to choose a context, the signers could compute $H(\text{pk}_R \parallel \text{cur_epoch})$, with the epoch duration itself being fixed over some reasonable interval so that all signers must issue their signatures within that epoch.

and $m := g^{1/(\kappa+\alpha)}$. This does not affect the issuance times or the presignature and final signatures compared to the above mentioned (non-threshold) NIBS construction. We also specify this in Table 1.

	# moves	Threshold	Recipient key size $ \text{pk}_R $	Communication		Signature size $ \sigma $
				$R \rightarrow S$	$S \rightarrow R$	
[VZK03] ⁵	2	✓	–	$1\mathbb{G} + 1\mathbb{F}$	$1\mathbb{G}$	$1\mathbb{G}$
[CKM ⁺ 23] ⁶	5	✓	–	$O(t\mathbb{F})$	$2\mathbb{G} + 4\mathbb{F}$	$1\mathbb{G} + 2\mathbb{F}$
[Han23]	1	✗	$1\mathbb{G}$	–	$2\mathbb{G} + 1\tilde{\mathbb{G}}$	$2\mathbb{G} + 1\tilde{\mathbb{G}}$
This work	1	✓	$1\mathbb{G} + 2\mathbb{F}$	–	$2\mathbb{G}$	$2\mathbb{G} + 3\mathbb{F}$

Table 1: Comparison of group-based non-interactive and/or threshold blind signature schemes. $R \rightarrow S$ denotes the communication from the recipient to the signer (similarly $S \rightarrow R$). $\mathbb{G}$, $\tilde{\mathbb{G}}$ denote the base groups, $\mathbb{F}$ the finite field elements and t is the threshold. Note that for type-3 pairing groups, the size of $\tilde{\mathbb{G}}$ in bits is at least 4 times bigger than the elements of $\mathbb{F}$.

Security of NITBS from PS signatures. Unforgeability of our scheme reduces to the security of the underlying (threshold variant of the) PS signature scheme; the reduction is straightforward because presignature generation is derived from the standard PS signing exponent with additional randomized masking that cancels at finalization. As indicated earlier, blindness can be proven almost identically to the non-threshold counterpart.

1.2 Related work

Blind signature schemes have been realized under a variety of assumptions. Constructions exist in the plain model [GRS⁺11, GG14, Gha17], though most practical schemes rely on idealized models [PS96, PS00b, Abe01, Bol03, BL13, FHS15, HKL19, KLR21, KLX22, CAHL⁺22, TZ22, CATZ24]. Generic approaches include those of Juels et al. [JLO97] and Fischlin [Fis06]. The recent interest in post-quantum secure cryptography has also led to several lines of work in lattice-based blind signature constructions [LNP22, dPK22, AKSY22, BLNS23, JS25].

Given the nature of our protocol, we focus our related work discussion below on threshold and non-interactive blind signatures and summarize some comparisons in Table 1.

Threshold blind signatures. The literature on threshold blind signatures is relatively sparse. [VZK03] built on [Bol03] to construct a two-move BLS-style scheme. Kuchta and Manulis [KM15] proposed a two-move protocol from bilinear pairings, though their first message is large due to two required NIZK proofs. Crites et al. [CKM⁺23] introduced a five-move protocol in pairing-free groups, obtained by compiling the [TZ22] blind signature scheme and achieving efficient verification. In a recent work, Lehmann et al. [LNÖ25] provided the first systematic study of unforgeability notions for threshold blind signatures, showing separations between existing schemes and the strongest definitions, and extending the [VZK03, CKM⁺23] constructions to achieve stronger guarantees. Coconut [SAB⁺19] is a multi-round protocol for threshold anonymous credentials. Our work has some structural similarities with Coconut as both schemes use PS signatures [PS16] to build a credential scheme. However the main challenge in our design stems from the non-interactive nature of our issuance.

⁵Satisfies the weakest notion of one-more unforgeability for threshold blind signatures (OM-Unf-0).

⁶Pairing free.

Non-interactivity. Hanzlik [Han23] introduced non-interactive blind signatures, giving a practical construction from signatures on equivalence classes [HS14, FHS19], proven secure in the generic group model. Baldimtsi et al. [BCGY24] strengthened the definitional framework and proposed new constructions, including a generic approach based on NIZKs. Hanzlik et al. [HPZ25] later gave a generic garbled-circuit-based construction supporting presignature issuance on RSA keys. Most recently, Baldimtsi et al. [BGY25] instantiated the [BCGY24] paradigm to obtain an efficient, strongly secure lattice-based NIBS scheme, and further extended the framework to enable batch issuance from a single presignature.

2 Preliminaries

Notation. We use λ to denote the security parameter. We use $\leftarrow \$$ to denote the output of a randomized algorithm and $\leftarrow$ to denote output of a deterministic algorithm. We denote the set of all positive integers up to n as $[n] := \{1, \dots, n\}$ and the set of all non-negative integers up to n as $[0, n] := \{0\} \cup [n]$. With slight abuse of notation, we also use $[x]_i$ to denote the i^{th} secret share of x , although the distinction will always be clear from context. $\mathbb{Z}$ is the ring of integers and $\mathbb{F}$ is a field. For any prime p , $\mathbb{Z}_p$ is the finite field of integers modulo p .

2.1 Signature schemes

A signature scheme over message space $\mathcal{M}$ consists of a tuple of polynomial-time algorithms (KeyGen, Sign, Verify) defined as follows:

- $\text{Setup}(1^\lambda) \rightarrow (\text{pk}, \text{sk})$. Given the security parameter $\lambda \in \mathbb{N}$, the probabilistic key generation algorithm outputs a public signature verification key $\text{pk} \in \mathcal{PK}$ and a secret signing key $\text{sk} \in \mathcal{SK}$.
- $\text{Sign}(\text{sk}, m) \rightarrow \sigma$. Given the signing key $\text{sk} \in \mathcal{SK}$ and a message $m \in \mathcal{M}$, the (possibly probabilistic) signature generation algorithm returns a signature $\sigma \in \Sigma$.
- $\text{Verify}(\text{pk}, m, \sigma) \rightarrow 1/0$. Given the public key $\text{pk} \in \mathcal{PK}$, a message $m \in \mathcal{M}$ and a signature σ , the deterministic signature verification algorithm outputs 1 (accept) or 0 (reject).

A signature scheme must satisfy the following properties:

- Correctness.** For every $\lambda \in \mathbb{N}$ and $m \in \mathcal{M}$,

$$\Pr \left[\text{Verify}(\text{pk}, m, \sigma) = 1 : \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \$ \text{KeyGen}(1^\lambda) \\ \sigma \leftarrow \$ \text{Sign}(\text{sk}, m) \end{array} \right].$$

- Existential unforgeability under chosen messages.** For every PPT adversary $\mathcal{A}$ there exists a negligible function $\epsilon(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\text{Verify}(\text{pk}, m^*, \sigma^*) = 1 : \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \$ \text{Setup}(1^\lambda) \\ (m^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sign}}}(\text{pk}) \end{array} \right]$$

where the signing oracle $\mathcal{O}_{\text{Sign}}$, on query m , outputs $\text{Sign}(\text{sk}, m)$. We say that $\mathcal{A}$ is admissible if it never queries m^* to $\mathcal{O}_{\text{Sign}}$.

2.2 Non-interactive zero knowledge proof systems

A non-interactive zero-knowledge (NIZK) proof system for relation $\mathcal{R}$ is a tuple of polynomial-time algorithms (Setup, Prove, Verify) defined as follows:

- $\text{Setup}(1^\lambda) \rightarrow \text{crs}$. The setup algorithm takes as input the security parameter $\lambda \in \mathbb{N}$, and outputs a common reference string crs .
- $\text{Prove}(\text{crs}, x, w) \rightarrow \pi$. The prover algorithm takes as input the crs , an instance $x \in \mathcal{L}_{\mathcal{R}}$, and a witness w . It outputs a proof π .
- $\text{Verify}(\text{crs}, x, \pi) \rightarrow 1/0$. The verification algorithm takes as input the crs , an instance x , and a proof π . It outputs either 1 (accept) or 0 (reject).

A NIZK proof system must satisfy the following properties:

- i. **Completeness.** For every $\lambda \in \mathbb{N}$, any instance and witness pair $(x, w) \in \mathcal{R}$,

$$\Pr \left[\text{Verify}(\text{crs}, x, \pi) = 1 : \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda) \\ \pi \leftarrow \text{Prove}(\text{crs}, x, w) \end{array} \right] = 1.$$

- ii. **Soundness.** For every stateful PPT adversary $\mathcal{A}$, there exists a negligible function $\epsilon(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{crs}, x, \pi) = 1 \\ \exists w : (x, w) \in \mathcal{R} \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda) \\ (x, \pi) \leftarrow \mathcal{A}(\text{crs}) \end{array} \right] \leq \epsilon(\lambda).$$

- iii. **Zero-knowledge.** There exists a *stateful* PPT simulator $\mathcal{S}$ such that for every *stateful* PPT adversary $\mathcal{A}$, there exists a negligible function $\epsilon(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \mathcal{A}(\pi_b) = b \wedge (x, w) \in \mathcal{R} : \begin{array}{l} b \leftarrow \{0, 1\} \\ \text{crs}_0 \leftarrow \text{Setup}(1^\lambda), \text{crs}_1 \leftarrow \mathcal{S}(1^\lambda) \\ (x, w) \leftarrow \mathcal{A}(\text{crs}_b) \\ \pi_0 \leftarrow \text{Prove}(\text{crs}_0, x, w) \\ \pi_1 \leftarrow \mathcal{S}(\text{crs}_1, x) \end{array} \right] \leq \frac{1}{2} + \epsilon(\lambda).$$

Argument of knowledge. A NIZK proof system Π is an argument of knowledge (NIZKAoK) if it additionally satisfies the following property:

- iv. **Knowledge soundness.** There exists a PPT extractor $\mathcal{E}$ such that for every *stateful* PPT adversary $\mathcal{A}$, there exists a negligible function $\epsilon(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{crs}, x, \pi) = 1 \\ \wedge (x, w) \notin \mathcal{R} \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda) \\ (x, \pi) \leftarrow \mathcal{A}(\text{crs}) \\ w \leftarrow \mathcal{E}(x, \pi) \end{array} \right] \leq \epsilon(\lambda).$$

Online extraction. A knowledge extractor $\mathcal{E}$ is called ‘online’ (also, ‘straight-line’) if it does not rewind the adversary.

2.3 Non-interactive blind signature schemes

We use the context-as-input model for non-interactive blind signatures proposed in [Han23], with the corrected blindness definitions from [BCGY24]⁷. Formally, a non-interactive blind signature (NIBS) scheme consists of a tuple of polynomial-time algorithms (Setup, KeyGen_S, KeyGen_R, Issue, Obtain, Verify) defined as follows:

- $\text{Setup}(1^\lambda) \rightarrow \text{pp}$. Given the security parameter $\lambda \in \mathbb{N}$, the probabilistic setup algorithm generates the collection of public parameters pp , which is an implicit input to the other algorithms.
- $\text{KeyGen}_S(\text{pp}) \rightarrow (\text{pk}, \text{sk})$. Given the public parameters pp , the probabilistic (signer) key generation algorithm outputs a public key $\text{pk} \in \mathcal{PK}$ and a secret key $\text{sk} \in \mathcal{SK}$.
- $\text{KeyGen}_R(\text{pp}) \rightarrow (\text{pk}_R, \text{sk}_R)$. Given the public parameters pp , the probabilistic recipient key generation algorithm outputs a $\text{pk}_R \in \mathcal{PK}_R$ and a secret key $\text{sk}_R \in \mathcal{SK}_R$.
- $\text{Issue}(\text{sk}, \text{pk}_R, c) \rightarrow \bar{\sigma}$. Given a signing key share $\text{sk} \in \mathcal{SK}$, a recipient public key $\text{pk}_R \in \mathcal{PK}_R$ and context $c \in \{0, 1\}^\lambda$, the probabilistic issuance algorithm outputs a presignature share $\bar{\sigma} \in \bar{\Sigma}$.
- $\text{Obtain}(\text{sk}_R, \text{pk}, c, \bar{\sigma}) \rightarrow (m, \sigma) / (\perp, \perp)$. Given the recipient secret key $\text{sk}_R \in \mathcal{SK}_R$, the public key $\text{pk} \in \mathcal{PK}$, context $c \in \{0, 1\}^\lambda$, and a presignature $\bar{\sigma} \in \bar{\Sigma}$, the probabilistic obtain algorithm outputs a message and signature pair $(m, \sigma) \in \mathcal{M} \times \Sigma$ or $(\perp, \perp)$.
- $\text{Verify}(\text{pk}, m, \sigma) \rightarrow 1/0$. Given the master public key $\text{mpk} \in \mathcal{PK}$, a message $m \in \mathcal{M}$ and a signature $\sigma \in \Sigma$, the deterministic signature verification algorithm outputs 1 (accept) or 0 (reject).

Furthermore, a NIBS scheme NIBS must satisfy the following properties:

- i. **Correctness.** For every $\lambda, c \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{pk}, m, \sigma) = 1 : \\ \text{pp} \leftarrow \$ \text{Setup}(1^\lambda) \\ (\text{pk}, \text{sk}) \leftarrow \$ \text{KeyGen}_S(\text{pp}) \\ (\text{pk}_R, \text{sk}_R) \leftarrow \$ \text{KeyGen}_R(\text{pp}) \\ \bar{\sigma} \leftarrow \$ \text{Issue}(\text{sk}, \text{pk}_R, c) \\ (m, \sigma) \leftarrow \$ \text{Obtain}(\text{sk}_R, \text{pk}, c, \bar{\sigma}) \end{array} \right] = 1.$$

- ii. **One-more unforgeability.** For every *stateful, admissible* PPT adversary $\mathcal{A}$, there exists a negligible function $\epsilon(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \bigwedge_{i \in [\ell+1]} \text{Verify}(\text{pk}, m_i, \sigma_i) = 1 \\ \bigwedge_{i \neq j \in [\ell+1]} m_i \neq m_j \end{array} : \begin{array}{l} \text{pp} \leftarrow \$ \text{Setup}(1^\lambda) \\ (\text{sk}, \text{pk}) \leftarrow \$ \text{KeyGen}_S(\text{pp}) \\ \{(m_i, \sigma_i)\}_{i=1}^{\ell+1} \leftarrow \mathcal{A}^{O_{\text{Issue}}}(\text{pp}, \text{pk}) \end{array} \right] \leq \epsilon(\lambda),$$

where oracle O_{Issue} takes as input a recipient's public key pk_R and context c , and outputs a presignature $\bar{\sigma}$ by running $\text{Issue}(\text{sk}, \text{pk}_R, c)$, and $\mathcal{A}$ is *admissible* iff $\mathcal{A}$ makes at most ℓ queries to O_{Issue} .

⁷[BCGY24] showed that the context-as-input and context-as-output models are essentially equivalent.

- iii. **Recipient blindness.** For every *stateful, admissible* PPT adversary $\mathcal{A}$, there exists a negligible function $\epsilon(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \mathcal{A}^{O_{\text{Obtain}}^R}(m_b, \sigma_b, m_{1-b}, \sigma_{1-b}) = b : \\ \text{pp} \leftarrow \text{Setup}(1^\lambda), b \leftarrow \{0, 1\}, \\ \forall \omega \in \{0, 1\} : (\text{sk}_{R_\omega}, \text{pk}_{R_\omega}) \leftarrow \text{KeyGen}_R(\text{pp}) \\ (\text{pk}, \{c_\omega, \tilde{\sigma}_\omega\}_{\omega \in \{0, 1\}}) \leftarrow \mathcal{A}^{O_{\text{Obtain}}^R}(\text{pp}, \text{pk}_{R_0}, \text{pk}_{R_1}) \\ \forall \omega \in \{0, 1\} : (m_\omega, \sigma_\omega) \leftarrow \text{Obtain}(\text{sk}_{R_\omega}, \text{pk}, c_\omega, \tilde{\sigma}_\omega) \end{array} \right] \leq \frac{1}{2} + \epsilon(\lambda) ,$$

where oracle O_{Obtain}^R , on query $(\omega, \text{pk}, \tilde{\sigma}, c)$, outputs $\text{Obtain}(\text{sk}_{R_\omega}, \text{pk}, c, \tilde{\sigma})$. We say that $\mathcal{A}$ is admissible iff:

- $\sigma_0, \sigma_1 \neq \perp$ (i.e., Obtain algorithm does not abort); and
 - $c_0 \neq c$ and $c_1 \neq c$ for any context c queried to O_{Obtain}^R .
- iv. **Context blindness.** For every *stateful, admissible* PPT adversary $\mathcal{A}$, there exists a negligible function $\epsilon(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \mathcal{A}^{O_{\text{Obtain}}^C}(m_b, \sigma_b, m_{1-b}, \sigma_{1-b}) = b : \\ \text{pp} \leftarrow \text{Setup}(1^\lambda), b \leftarrow \{0, 1\} \\ (\text{sk}_R, \text{pk}_R) \leftarrow \text{KeyGen}_R(\text{pp}) \\ (\text{pk}, \{c_\omega, \tilde{\sigma}_\omega\}_{\omega \in \{0, 1\}}) \leftarrow \mathcal{A}^{O_{\text{Obtain}}^C}(\text{pp}, \text{pk}_R) \\ \forall \omega \in \{0, 1\} : (m_\omega, \sigma_\omega) \leftarrow \text{Obtain}(\text{sk}_{R_\omega}, \text{pk}, c_\omega, \tilde{\sigma}_\omega) \end{array} \right] \leq \frac{1}{2} + \epsilon(\lambda) ,$$

where oracle $\mathcal{A}^{O_{\text{Obtain}}^C}$, on query $(\text{pk}, \tilde{\sigma}, c)$, outputs $\text{Obtain}(\text{sk}_R, \text{pk}, c, \tilde{\sigma})$. We say that $\mathcal{A}$ is admissible iff:

- $\sigma_0, \sigma_1 \neq \perp$ (i.e., Obtain algorithm does not abort); and
- $c_0 \neq c$ and $c_1 \neq c$ for any context c queried to O_{Obtain}^C .

2.4 Threshold secret sharing

A (t, n) -threshold secret sharing scheme over some field $\mathcal{X}$ consists of a tuple of polynomial-time algorithms (Share, Combine) defined as follows:

- Share $_{t,n}(x) \rightarrow [x]_{i \in [n]}$. Given an element $x \in \mathcal{X}$, the probabilistic secret share generation algorithm outputs a collection of n elements $\{[x]_i \in \mathcal{X} : \forall i \in [n]\}$ denoted $[x]_{i \in [n]}$.
- Combine $_{t,n}(\mathcal{S}, [x]_{i \in \mathcal{S}}) \rightarrow x / \perp$. Given a set $\mathcal{S} \subset [n]$ of indices, and a collection of $|\mathcal{S}|$ shares $[x]_i \in \mathcal{X} : \forall i \in \mathcal{S}$, the deterministic share recombination algorithm outputs x or $\perp$.

Furthermore, a (t, n) -threshold secret sharing scheme must satisfy the following properties:

- (i) **Correctness.** For every $1 \leq t \leq n \in \mathbb{N}$, and any subset $\mathcal{S} \subset [n]$ such that $|\mathcal{S}| = t$,

$$\Pr[\text{Combine}_{t,n}(\mathcal{S}, [x]_{i \in \mathcal{S}}) = x : [x]_{i \in [n]} \leftarrow \text{Share}_{t,n}(x)] = 1 .$$

(ii) **Robustness.** For every $t \leq n \in \mathbb{N}$, and any probabilistic adversary $\mathcal{A}$

$$\Pr \left[\begin{array}{l} \text{Combine}_{t,n}(\mathcal{S}, [x]_{i \in \mathcal{S}}) = \perp \\ \wedge |\mathcal{S}| < t \end{array} : \begin{array}{l} [x]_{i \in [n]} \leftarrow \text{Share}_{t,n}(x) \\ \mathcal{S} \leftarrow \mathcal{A}([x]_{i \in [n]}) \end{array} \right] = 1.$$

Shamir's secret sharing. Let p be a prime power with $p > n$ and let $\mathbb{Z}_p$ denote the finite field of order p . For integers $1 \leq t \leq n$ and a secret $s \in \mathbb{Z}_p$ the (t, n) -Shamir secret sharing scheme samples $a_1, \dots, a_{t-1} \leftarrow \mathbb{Z}_p$ uniformly and defines the random polynomial $f(x) = s + \sum_{i=1}^{t-1} a_i x^i \in \mathbb{Z}_p[x]$ so that we can define the i^{th} share $[s]_i := f(i)$. Importantly, any set of t shares determines f uniquely by polynomial interpolation, and in particular the secret is recovered as the constant term $s = f(0)$ via the Lagrange reconstruction formula $s = f(0) = \sum_{i=1}^t y_i \Lambda_i$, where $\Lambda_i := \prod_{k \in [t] \setminus \{i\}} \frac{k}{k-i}$ is the i^{th} Lagrange coefficient.

2.5 Bilinear groups

Definition 2.1 (Pairing friendly groups). Let $\mathbb{G}$ and $\tilde{\mathbb{G}}$ be two groups of the same prime order p with the generators g and $\tilde{g}$, respectively. Also, let $\mathbb{Z}_p$ be the field of order p . Then, $\mathbb{G}$ and $\tilde{\mathbb{G}}$ are *pairing friendly* if there exists an efficiently computable bilinear pairing, $e : \mathbb{G} \times \tilde{\mathbb{G}} \rightarrow \hat{\mathbb{G}}$, satisfying the following properties:

- **Bilinearity:** $\forall h \in \mathbb{G}, \tilde{h} \in \tilde{\mathbb{G}}, x, y \in \mathbb{Z}_p$,

$$e(h^x, \tilde{h})^y = e(h, \tilde{h}^y)^x = e(h, \tilde{h})^{xy}.$$

- **Non-degeneracy:** $e(g, \tilde{g}) \neq 1_{\hat{\mathbb{G}}}$.

Bilinear pairings can be of a few types depending on whether there is an efficiently computable homomorphism from $\mathbb{G}$ to $\tilde{\mathbb{G}}$ in both directions (Type 1), only one direction (Type 2), or in neither direction (Type 3). Type 3 pairings are the most efficient setting for a relevant security parameter and they are commonly deployed.

2.6 Hardness assumptions

Definition 2.2 (Co-discrete Logarithm Problem). Let $(\mathbb{G}, \tilde{\mathbb{G}})$ be pairing friendly groups of order p with a bilinear pairing $e : \mathbb{G} \times \tilde{\mathbb{G}} \rightarrow \hat{\mathbb{G}}$. For $x \leftarrow \mathbb{Z}_p$ the co-discrete logarithm problem asks that given $(p, g, \tilde{g}, X := g^x, \tilde{X} := \tilde{g}^x)$, find and output x .

Definition 2.3 (One-more Co-discrete Logarithm Problem). Let $(\mathbb{G}, \tilde{\mathbb{G}})$ be pairing friendly groups of order p with a bilinear pairing $e : \mathbb{G} \times \tilde{\mathbb{G}} \rightarrow \hat{\mathbb{G}}$. For $x \leftarrow \mathbb{Z}_p$ the one-more co-discrete logarithm problem asks that given $(p, g, \tilde{g})$, oracle $\mathcal{O}$ that outputs tuples $(g^{t_i}, \tilde{g}^{t_i})$ and oracle $\mathcal{O}_{\text{dlog}}$ that on input $Y \in \mathbb{G}$ (respectively $\tilde{Y} \in \tilde{\mathbb{G}}$) outputs y such that $Y = g^y$ (respectively $\tilde{Y} = \tilde{g}^y$), find and output distinct $t_1, \dots, t_{\ell+1}$, where for each $i \in [\ell+1]$ there exists an output of oracle $\mathcal{O}$ such that $(g^{t_i}, \tilde{g}^{t_i})$, while making at most ℓ queries to $\mathcal{O}_{\text{dlog}}$.

Definition 2.4 (Strong Decisional Diffie-Hellman Problem (sDDH)). Let $\mathbb{G}$ be a group of order p . For $x, y \leftarrow \mathbb{Z}_p$, and $g \in \mathbb{G}$, the decisional Diffie-Hellman problem asks to distinguish, for a randomly chosen $T \in \mathbb{G}$, the distributions $\mathcal{D}_0$ and $\mathcal{D}_1$, where:

$$\mathcal{D}_0 := (p, g, g^x, g^{1/x}, g^y, g^{xy}) \quad \mathcal{D}_1 := (p, g, g^x, g^{1/x}, g^y, T).$$

Definition 2.5 (Decisional q -Diffie-Hellman Inverse Problem (qDDHI)). Let $\mathbb{G}$ be a group of order p . For $x \leftarrow \mathbb{Z}_p$, and $g \in \mathbb{G}$, the decisional q -Diffie-Hellman inverse problem asks to distinguish, for a randomly chosen $T \in \mathbb{G}$, the distributions $\mathcal{D}_0$ and $\mathcal{D}_1$, where:

$$\mathcal{D}_0 := (p, g, g^x, g^{x^2}, \dots, g^{x^q}, g^{1/x}) \quad \mathcal{D}_1 := (p, g, g^x, g^{x^2}, \dots, g^{x^q}, T).$$

Definition 2.6 (PS Problem [PS16, Assumption 2]). Let $(\mathbb{G}, \tilde{\mathbb{G}})$ be pairing friendly groups of order p with a bilinear pairing $e : \mathbb{G} \times \tilde{\mathbb{G}} \rightarrow \tilde{\mathbb{G}}$. For $x, y \leftarrow \mathbb{Z}_p$ the PS problem asks that given $(p, g, \tilde{g}, e, \tilde{X} := \tilde{g}^x, \tilde{Y} := \tilde{g}^y)$ and an oracle $\mathcal{O}$ that on input $\mu \in \mathbb{Z}_p$ outputs $(h, h^{x+y\mu})$, find a $(\mu^*, h^*, (h^*)^{x+y\mu^*})$ triple such that μ^* is never queried to $\mathcal{O}$ and $h^* \neq 1_{\mathbb{G}}$.

Notably, the PS problem is reducible to the hardness of the co-discrete logarithm problem, as we explain next.

Lemma 2.7. *In the algebraic group model, the PS assumption holds under the co-discrete logarithm assumption.*

Sketch. Complete proofs of the security of the PS assumption in the algebraic group model can be found in [KSAP23] and [BFL20]. Here, we sketch the proof idea. Because the adversary is algebraic, it must output a representation for the final forgery (μ^*, h^*, H^*) . Thus, we can define $a_x, a_y, a_R, b_x, b_y, b_R$, such that:

$$h^* = g^{a_x \cdot x + a_y \cdot y + a_R} \quad \text{and} \quad H^* = g^{b_x \cdot x + b_y \cdot y + b_R},$$

whereby the validity of the output we know that

$$(a_x \cdot x + a_y \cdot y + a_R) \cdot (x + \mu^* \cdot y) = b_x \cdot x + b_y \cdot y + b_R.$$

The proof then focuses on three distinct cases, where the reduction picks different targets for a co-discrete logarithm instance. It either tries to use the adversary to compute x , where $\tilde{X}$ is set to the co-discrete logarithm instance. Alternatively, it targets y and appropriately sets $\tilde{Y}$. The last target, and a way for an adversary to break the PS assumption, is to compute the discrete logarithm of h_i to base g , where $(h_i, h_i^{x+y\mu_i})$ is a response from oracle $\mathcal{O}$. However, note that the reduction can use this to solve the co-discrete logarithm by setting h_i . $\square$

We also recount to the reader that in the algebraic group model, an adversary is restricted to performing group operations exclusively on the group elements it has been given.

Definition 2.8 (Algebraic Group Model (AGM) [FKL18]). For every group element that an adversary outputs as the challenge or as part of a query, it must output its *representation* in terms of the group elements it has previously been provided by the challenger. For instance, when an adversary, having seen group elements $R_0, R_1, \dots, R_n$, outputs a group element P , it must also output a vector $(a_0, a_1, \dots, a_n) \in \mathbb{Z}_p^n$ such that $P = \prod_{i \in [0, n]} R_i^{a_i}$.

Further, we call an adversary satisfying these conditions, an *algebraic* adversary.

3 Non-interactive Threshold Blind Signatures

In this section we formalize non-interactive threshold blind signatures (NITBS) and give the precise security notions tailored to the non-interactive setting. In interactive protocols “non-interactivity” is sometimes taken to mean a single round of communication (recipient $\rightarrow$ signers $\rightarrow$ recipient) [Pas11, LNÖ25]. Here we mean *strict* non-interactivity, i.e., each signer produces a single presignature share that the recipient finalizes locally. This single-message structure simplifies the security model compared to recent interactive treatments (e.g. [LNÖ25]).

Due to the inherent non-interactivity of our scheme, we are able to avoid all session bookkeeping – no rounds, no session identifiers and no session state. In particular, this lets us state a simple and intuitive one-more unforgeability notion wherein an efficient adversary can adaptively corrupt any subset of signers (learning their secret keys) and may request presignature shares from honest signers. As a result, we can model scenarios where an adversary, having at least t presignatures per (pk_R, c) pair by making presignature queries and by corrupting up to $(t-1)$ signers, cannot come up with more than the expected number of forgeries. This notion corresponds closely to the one-more unforgeability notion OM-Unf-1 from [LNÖ25].

We also describe a weaker one-more unforgeability notion. Informally, in this weak model each time the adversary makes a presignature query on some (pk_R, c) pair, the challenger returns the presignature shares of all non-corrupted signers (i.e., exactly what an honest recipient would obtain). We therefore treat such a query as if the adversary already possesses a usable presignature on (pk_R, c) ; consequently the adversary’s count of legitimately obtained presignatures ℓ increases immediately, and the one-more condition requires that the adversary cannot output more than ℓ valid finalized signatures⁸. The contrast with the previous stronger notion is important: in the strong model a presignature query does not automatically increase ℓ unless the adversary actually obtains enough signer shares (including shares obtained via corruption). The weak notion therefore corresponds roughly to the OM-Unf-0 variant of one-more unforgeability in [LNÖ25], while the strong notion corresponds to their OM-Unf-1.

It is worth noting that these two definitions completely capture all possible issuing-oracle behaviors in our framework, since we do not need to model sessions or maintain state across protocol runs. Importantly, as our threshold blind signature construction is non-interactive, the four separate one-more unforgeability variants introduced in [LNÖ25] for interactive threshold blind signatures are not required. For the corruption model we adopt the strongest (adaptive) notion; nonetheless, the definitions can be rephrased to accommodate an adversary that corrupts signers selectively. More details are given below.

Definition 3.1. A non-interactive threshold blind signature scheme over message space $\mathcal{M}$ is a tuple of polynomial time algorithms (Setup, KeyGen_S, KeyGen_R, Issue, Obtain) defined as follows:

- Setup(1^λ) $\rightarrow$ pp. Given the security parameter λ , the probabilistic setup algorithm generates the collection of public parameters pp, which is an implicit input to the other algorithms.
- KeyGen_S(pp, t, n) $\rightarrow$ (mpk, $\{sk_i\}_{i \in [n]}$). Given the public parameters pp, the minimum threshold weight t and the number of signers n , the probabilistic (signer) key generation algorithm outputs a master public key $mpk \in \mathcal{PK}$ and a set of secret signing key shares $(sk_1, \dots, sk_n) \in \mathcal{SK}^n$. We assume without loss of generality that mpk implicitly contains t and n .

⁸The weaker notion could, for instance, be useful for blockchain applications where the signatures may be posted on-chain at once.

- $\text{KeyGen}_R(pp) \rightarrow (pk_R, sk_R)$. Given the public parameters pp , the probabilistic recipient key generation algorithm outputs a $pk_R \in \mathcal{PK}_R$ and a secret key $sk_R \in \mathcal{SK}_R$.
- $\text{Issue}(sk, pk_R, c) \rightarrow \bar{\sigma}$. Given a signing key share $sk \in \mathcal{SK}$, a recipient public key $pk_R \in \mathcal{PK}_R$ and context $c \in \{0, 1\}^\lambda$, the probabilistic issuance algorithm outputs a presignature share $\bar{\sigma} \in \bar{\Sigma}$.
- $\text{Obtain}(sk_R, mpk, \mathcal{S}, c, \{\bar{\sigma}_i\}_{i \in \mathcal{S}}) \rightarrow (m, \sigma) / (\perp, \perp)$. Given the recipient secret key $sk_R \in \mathcal{SK}_R$, the master public key $mpk \in \mathcal{PK}$, a set $\mathcal{S} \subseteq [n]$ of indices, context $c \in \{0, 1\}^\lambda$, and a collection of presignature shares $\bar{\sigma}_i \in \bar{\Sigma}$ for every $i \in \mathcal{S}$, the probabilistic obtain algorithm outputs a message and signature pair $(m, \sigma) \in \mathcal{M} \times \Sigma$ or $(\perp, \perp)$.
- $\text{Verify}(mpk, m, \sigma) \rightarrow 1/0$. Given the master public key $mpk \in \mathcal{PK}$, a message $m \in \mathcal{M}$ and a signature $\sigma \in \Sigma$, the deterministic signature verification algorithm outputs 1 (accept) or 0 (reject).

It must further satisfy the following properties:

Definition 3.2 (Correctness). For all $\lambda, t, n \in \mathbb{N}$, such that $t \leq n$, all $c \in \{0, 1\}^\lambda$, and all $\mathcal{S} \subseteq [n]$ satisfying $|\mathcal{S}| \geq t$, a non-interactive threshold blind signature scheme NITBS satisfies *correctness* if

$$\Pr \left[\begin{array}{l} \text{Verify}(mpk, m, \sigma) = 1 : \\ \begin{array}{l} pp \leftarrow \$ \text{Setup}(1^\lambda) \\ (mpk, \{sk_i\}_{i \in [n]}) \leftarrow \$ \text{KeyGen}_S(pp, t, n) \\ (pk_R, sk_R) \leftarrow \$ \text{KeyGen}_R(pp) \\ \forall i \in \mathcal{S} : \bar{\sigma}_i \leftarrow \$ \text{Issue}(sk_i, pk_R, c) \\ (m, \sigma) \leftarrow \$ \text{Obtain}(sk_R, mpk, \mathcal{S}, \{c_i, \bar{\sigma}_i\}_{i \in \mathcal{S}}) \end{array} \end{array} \right] = 1.$$

Definition 3.3 (Efficiency). For all $\lambda, n \in \mathbb{N}$, a non-interactive threshold blind signature scheme, NITBS, is *efficient* if $|m, \sigma| = O_\lambda(1)$, i.e., the final message and signature sizes are independent of n .

Game OM-Unf _{NITBS, A} (λ)	Oracle $\mathcal{O}_{\text{Issue}}(i, pk_R, c)$
1: $pp \leftarrow \$ \text{Setup}(1^\lambda), \mathcal{C} := \emptyset$	1: if $i \notin [n] \setminus \mathcal{C}$ then return $\perp$
2: $(mpk, \{sk_i\}_{i \in [n]}) \leftarrow \$ \text{KeyGen}_S(pp, t, n)$	2: if $Q_{pk_R, c}$ is uninitialized then
3: $\{m_j, \sigma_j\}_{j \in [\ell]} \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Issue}}, \mathcal{O}_{\text{Corrupt}}}(pp, mpk)$	3: $Q_{pk_R, c} := \emptyset$
4: Let $L = \{(pk_R, c) : Q_{pk_R, c} \cup \mathcal{C} \geq t+1\}$	4: $Q_{pk_R, c} := Q_{pk_R, c} \cup \{i\}$
5: if $\ell \leq L \vee \bigvee_{1 \leq j < j' \leq \ell} (m_j = m_{j'})$	5: return $\text{Issue}(sk_i, pk_R, c)$
$\bigvee_{j \in [\ell]} \text{Verify}(mpk, m_j, \sigma_j) \neq 1$ then	Oracle $\mathcal{O}_{\text{Corrupt}}(i)$
6: reject	1: if $i \notin [n] \setminus \mathcal{C} \vee$
7: accept	$ \mathcal{C} \geq t-1$ then
	2: return $\perp$
	3: $\mathcal{C} := \mathcal{C} \cup \{i\}$
	4: return sk_i

Figure 1: The one-more unforgeability experiment for NITBS.

Definition 3.4 (One-more unforgeability). For all $\lambda, n, t \in \mathbb{N}$ with $t \leq n$, let

$$\text{Adv}_{\text{NITBS}, \mathcal{A}}^{\text{OM-Unf}} = \Pr [\text{OM-Unf}_{\text{NITBS}, \mathcal{A}}(\lambda) = \text{accept}]$$

be the advantage of an adversary $\mathcal{A}$ in the experiment defined in Figure 1. We say a non-interactive threshold blind signature scheme, NITBS, is *one-more unforgeable* if for any PPT adversary $\mathcal{A}$ this advantage is negligible.

Game R-Blind _{NITBS, A} (λ)	Game C-Blind _{NITBS, A} (λ)
1: $\text{pp} \leftarrow \text{Setup}(1^\lambda), b \leftarrow \{0, 1\}$ 2: foreach $\omega \in \{0, 1\}$ do 3: $(\text{pk}_{R_\omega}, \text{sk}_{R_\omega}) \leftarrow \text{KeyGen}_R(\text{pp})$ 4: $\left(\text{mpk}, \left\{ \begin{smallmatrix} c_\omega, \\ \{\tilde{\sigma}_{\omega, i}\}_{i \in \mathcal{S}_\omega} \end{smallmatrix} \right\}_{\omega \in \{0, 1\}} \right) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Obtain}}^R}(\text{pk}_{R_0}, \text{pk}_{R_1})$ 5: foreach $\omega \in \{0, 1\}$ do 6: if $c_\omega \in Q$ then 7: reject 8: $(m_\omega, \sigma_\omega) \leftarrow \text{Obtain} \left(\begin{smallmatrix} \text{sk}_{R_\omega}, \text{mpk}, \mathcal{S}_\omega, \\ c_\omega, \{\tilde{\sigma}_{\omega, i}\}_{i \in \mathcal{S}_\omega} \end{smallmatrix} \right)$ 9: if $\text{Verify}(\text{mpk}, m_\omega, \sigma_\omega) \neq 1$ then 10: reject 11: $b^* \leftarrow \mathcal{A}(m_b, \sigma_b, m_{1-b}, \sigma_{1-b})$ 12: if $b^* = b$ then 13: accept 14: reject Oracle $\mathcal{O}_{\text{Obtain}}^R(\omega, \text{mpk}, \mathcal{S}, c, \{\tilde{\sigma}_i\}_{i \in \mathcal{S}})$ 1: if Q is uninitialized then 2: $Q := \emptyset$ 3: $Q := Q \cup \{c\}$ 4: return $\text{Obtain}(\text{sk}_{R_\omega}, \text{mpk}, \mathcal{S}, c, \{\tilde{\sigma}_i\}_{i \in \mathcal{S}})$	1: $\text{pp} \leftarrow \text{Setup}(1^\lambda), b \leftarrow \{0, 1\}$ 2: $(\text{pk}_R, \text{sk}_R) \leftarrow \text{KeyGen}_R(\text{pp})$ 3: $\left(\text{mpk}, \left\{ \begin{smallmatrix} c_\omega, \\ \{\tilde{\sigma}_{\omega, i}\}_{i \in \mathcal{S}_\omega} \end{smallmatrix} \right\}_{\omega \in \{0, 1\}} \right) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Obtain}}^C}(\text{pk}_R)$ 4: foreach $\omega \in \{0, 1\}$ do 5: if $c_\omega \in Q$ then 6: reject 7: $(m_\omega, \sigma_\omega) \leftarrow \text{Obtain} \left(\begin{smallmatrix} \text{sk}_R, \text{mpk}, \mathcal{S}_\omega, \\ c_\omega, \{\tilde{\sigma}_{\omega, i}\}_{i \in \mathcal{S}_\omega} \end{smallmatrix} \right)$ 8: if $\text{Verify}(\text{mpk}, m_\omega, \sigma_\omega) \neq 1$ then 9: reject 10: $b^* \leftarrow \mathcal{A}(m_b, \sigma_b, m_{1-b}, \sigma_{1-b})$ 11: if $b^* = b$ then 12: accept 13: reject Oracle $\mathcal{O}_{\text{Obtain}}^C(\text{mpk}, \mathcal{S}, c, \{\tilde{\sigma}_i\}_{i \in \mathcal{S}})$ 1: if Q is uninitialized then 2: $Q := \emptyset$ 3: $Q := Q \cup \{c\}$ 4: return $\text{Obtain}(\text{sk}_R, \text{mpk}, \mathcal{S}, c, \{\tilde{\sigma}_i\}_{i \in \mathcal{S}})$

Figure 2: The recipient (left) and context (right) blindness experiments for NITBS.

Definition 3.5 (Blindness). For all $\lambda, n, t \in \mathbb{N}$ with $t \leq n$, let

$$\text{Adv}_{\text{NITBS}, \mathcal{A}}^{\text{R-Blind}} = \Pr [\text{R-Blind}_{\text{NITBS}, \mathcal{A}}(\lambda) = \text{accept}]$$

be the advantage of a *stateful* adversary $\mathcal{A}$ in the experiment defined in Figure 2 (left), and

$$\text{Adv}_{\text{NITBS}, \mathcal{A}}^{\text{C-Blind}} = \Pr [\text{C-Blind}_{\text{NITBS}, \mathcal{A}}(\lambda) = \text{accept}]$$

be its advantage in the experiment defined in Figure 2 (right). We say a non-interactive threshold blind signature scheme, NITBS, satisfies *blindness* if for any PPT adversary $\mathcal{A}$ each of these advantages is at most $1/2 + \epsilon(\lambda)$, for some negligible function $\epsilon(\cdot)$.

For completeness, we also describe the weaker notion of one-more unforgeability, which roughly corresponds to the notion of OM-Unf-0 in threshold (interactive) blind-signatures.

Definition 3.6 (Weak one-more unforgeability). For all $\lambda, n, t \in \mathbb{N}$ with $t \leq n$

$$\text{Adv}_{\text{NITBS}, \mathcal{A}}^{\text{wOM-Unf}} = \Pr[\text{wOM-Unf}_{\text{NITBS}, \mathcal{A}}(\lambda) = \text{accept}]$$

denotes the advantage of an adversary $\mathcal{A}$ in the experiment defined in Figure 3. We say that a non-interactive threshold blind signature scheme, NITBS, is *weakly one-more unforgeable* if for any PPT adversary $\mathcal{A}$ this advantage is negligible.

Game $\text{wOM-Unf}_{\text{NITBS}, \mathcal{A}}(\lambda)$	Oracle $\mathcal{O}_{\text{Issue}}(\text{pk}_R, c)$
1: $\text{pp} \leftarrow \text{Setup}(1^\lambda), \mathcal{C} := \emptyset$	1: if L is uninitialized then
2: $(\text{mpk}, \{\text{sk}_i\}_{i \in [n]}) \leftarrow \text{KeyGen}_S(\text{pp}, t, n)$	2: $L := 0$
3: $\{m_j, \sigma_j\}_{j \in [\ell]} \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Issue}}, \mathcal{O}_{\text{Corrupt}}}(\text{pp}, \text{mpk})$	3: $L := L + 1$
4: if $\ell \leq L \vee \bigvee_{1 \leq j < j' \leq \ell} (m_j = m_{j'})$ $\bigvee_{j \in \ell} \text{Verify}(\text{mpk}, m_j, \sigma_j) \neq 1$ then	4: foreach $i \in [n] \setminus \mathcal{C}$ do
5: reject	5: $\tilde{\sigma}_i \leftarrow \text{Issue}(\text{sk}_i, \text{pk}_R, c)$
6: accept	6: return $\{\tilde{\sigma}_i\}_{i \in [n] \setminus \mathcal{C}}$

Figure 3: The weak one-more unforgeability experiment for NITBS.

4 NIBS from PS Signatures

In this section we describe our non-interactive (non-threshold) blind signature construction based on PS signatures. As discussed in the introduction this construction is of independent interest, but also a stepping stone towards building our threshold NIBS. We use PS signatures in a non-black-box way, and hence, the construction is described semi-generically. In particular, we make black-box use of a non-interactive proof system with efficient instantiations presented later in this section. Note that some of the described algorithms closely resemble those for the standard PS signature, i.e., the issuer's public key generation is simply the PS signature generation algorithm, however additional structure is introduced during the signature issuance stage. Nevertheless, the unforgeability of our non-interactive blind signature scheme is reducible to the security of the PS signature scheme. In fact, we actually show that even a weaker assumption is sufficient.

Construction 1. Our construction requires hash functions $H_G : \{0, 1\}^\lambda \rightarrow \mathbb{G}$ and $H_Z : \{0, 1\}^\lambda \rightarrow \mathbb{Z}_p^*$. We will also make use of two non-interactive proof systems $\Pi = (\Pi.\text{Setup}, \Pi.\text{Prove}, \Pi.\text{Verify})$, for languages $\mathcal{L}_R$

and $\mathcal{L}_{PS}$ as described.

Language $\mathcal{L}_R$

Instance: Each instance x is interpreted as group elements $(\phi_1, \phi_2) \in \mathbb{G}^2$.

Witness: Witness w consists of scalar $\alpha \in \mathbb{Z}_p^*$.

Membership: w is a valid witness for x if:

$$\phi_2^\alpha = \phi_1$$

Language $\mathcal{L}_{PS}$

Instance: Each instance x is interpreted as the issuer's public key $pk = (\tilde{X}, \tilde{Y}_1, \tilde{Y}_2) \in \tilde{\mathbb{G}}^3$, a group element $m \in \tilde{\mathbb{G}}$ and a PS signature $\Psi = (\psi_1, \psi_2) \in \mathbb{G}^2$.

Witness: Witness w consists of scalars $(\kappa, \alpha) \in (\mathbb{Z}_p^*)^2$.

Membership: w is a valid witness for x if the following are satisfied:

$$\begin{aligned} e(\psi_1, \tilde{X}) \cdot e(\psi_1, \tilde{Y}_1)^\kappa \cdot e(\psi_1, \tilde{Y}_2)^\alpha &= e(\psi_2, \tilde{g}) \quad \triangleright \quad \Phi \text{ is a valid PS signature for the message } (\kappa, \alpha) \\ \wedge m^{\kappa+\alpha} &= g_2 \quad \triangleright \quad m \text{ is a valid NIBS message constructed from } (\kappa, \alpha) \end{aligned}$$

The language will be evident from the statement used in $\Pi.\text{Prove}$ and $\Pi.\text{Verify}$. Moreover, we will use different common reference strings crs_R and crs_{PS} to distinguish those two different proof systems.

Setup. The public parameters are given by the tuple $pp = (e, p, \mathbb{G}, \tilde{\mathbb{G}}, \hat{\mathbb{G}}, g, g_1, g_2, \tilde{g}, \tilde{g}_1, \tilde{g}_2, \text{crs}_R, \text{crs}_{PS})$, where $e: \mathbb{G} \times \tilde{\mathbb{G}} \rightarrow \hat{\mathbb{G}}$ is a bilinear pairing of type 3, g, g_1, g_2 are generator of $\mathbb{G}$ and $\tilde{g}$ is a generator of $\tilde{\mathbb{G}}$, all of order p ; and the setup algorithm generates the common reference strings crs_R and crs_{PS} for the proof systems for languages $\mathcal{L}_R$ and $\mathcal{L}_{PS}$, respectively. It is worth noting that, depending on the instantiation of the proof systems, the setup algorithm can be transparent, i.e., no trusted party may be needed to run this setup algorithm.

Key generation. The signer's key generation algorithm samples a random $(x, y_1, y_2) \leftarrow (\mathbb{Z}_p^*)^3$ and computes public key $pk := (Y_1 := g^{y_1}, Y_2 := g^{y_2}, \tilde{X} := \tilde{g}^x, \tilde{Y}_1 := \tilde{g}^{y_1}, \tilde{Y}_2 := \tilde{g}^{y_2})$, while keeping $sk := (x, y_1, y_2)$ as the secret key. The recipient's key generation algorithm samples a random value $sk_R \leftarrow \mathbb{Z}_p^*$ as the secret signing key, and sets the public key $pk_R := (g_1^{1/sk_R}, \pi_R)$, where π_R is a proof of possession of the secret key.

KeyGen _S (pp)	KeyGen _R (pp)
1: $(x, y_1, y_2) \leftarrow (\mathbb{Z}_p^*)^3$	1: $\alpha \leftarrow \mathbb{Z}_p^*$
2: $\tilde{X} := \tilde{g}^x, \tilde{Y}_1 := \tilde{g}^{y_1}, \tilde{Y}_2 := \tilde{g}^{y_2}$	2: $\pi_R \leftarrow \Pi.\text{Prove} \left(\begin{array}{l} \text{crs}_R, x := (g_1, g_1^{1/\alpha}) \\ w := \alpha \end{array} \right)$
3: return $pk := (\tilde{X}, \tilde{Y}_1, \tilde{Y}_2),$ $sk := (x, y_1, y_2)$	3: return $pk_R := (g_1^{1/\alpha}, \pi_R),$ $sk_R := \alpha$

Presignature issuance and signature generation. We detail the presignature issuance and signature generation protocols in Figure 4. To issue a presignature, the issuer first verifies the recipient's public key by verifying the proof π_R using $\Pi.\text{Verify}(\text{crs}_R, x := (g_1, pk_R'), \pi_R)$, where $pk_R = (pk_R', \pi_R)$. In case the proof is invalid, the issuer aborts the issuance. Otherwise, the issuer picks a random $r \leftarrow \mathbb{Z}_p^*$ and computes the generator $h := H_{\mathbb{G}}(pk_R, c)$ using the recipient's public key pk_R and nonce c . Finally, the presignature $\bar{\sigma}$ consist of values (A, B) , where $A := h^{x+y_1\kappa} \cdot g_1^r$ and $B := (pk_R')^{-r} \cdot h^{y_2}$ for $\kappa = H_{\mathbb{Z}}(pk_R, c)$.

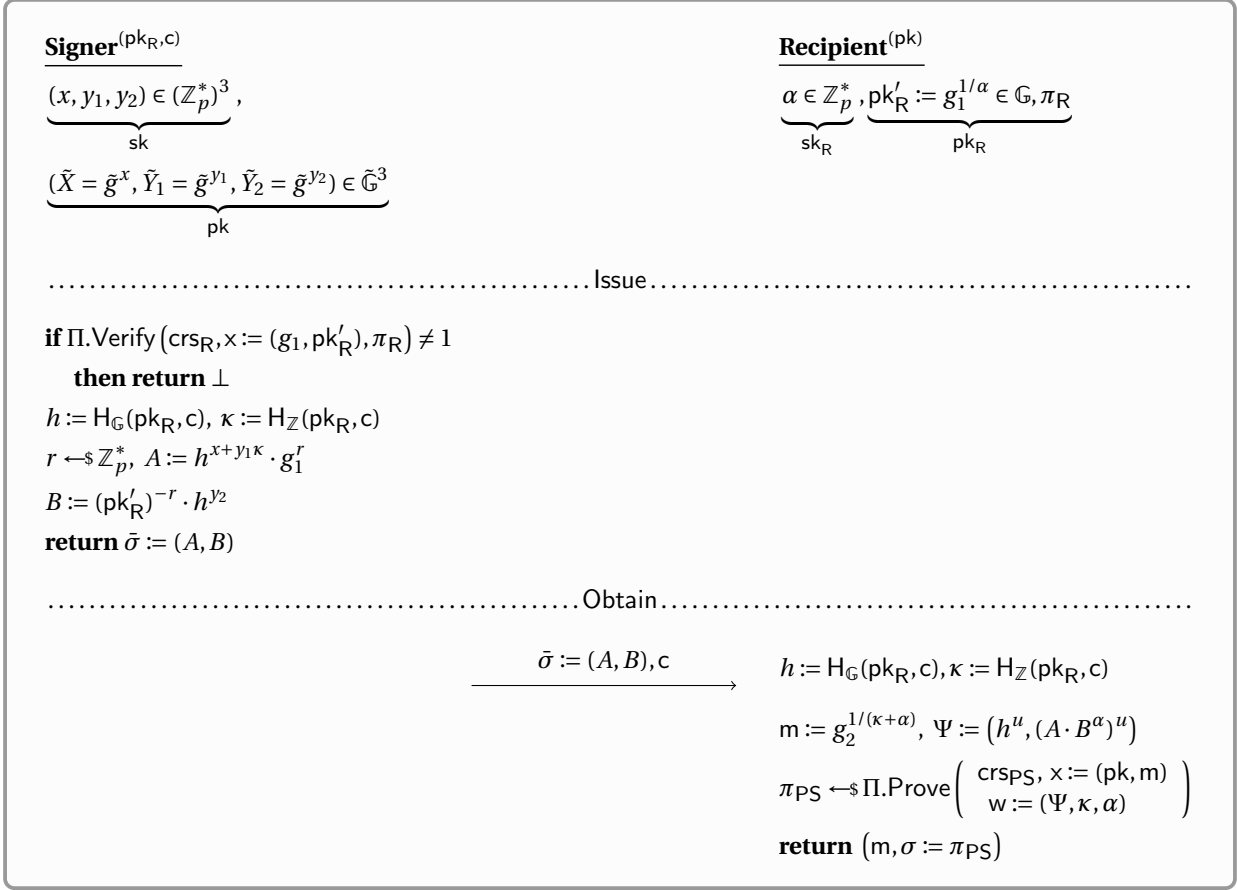


Figure 4: Presignature issuance and blind signature generation for construction 1.

To obtain a blind signature, the recipient recomputes the generator $h := H_{\mathbb{G}}(\text{pk}_R, c)$. It then sets $\Psi := (h^\rho, (A \cdot B^\alpha)^\rho)$, where $\text{sk}_R = \alpha$ and $\rho \leftarrow \$\mathbb{Z}_p^*$. It is worth reminding the reader that Ψ is a PS signature under the message vector (κ, α) . In the final step, the recipient computes the message for the blind signature as $m = g_2^{1/(\kappa+\alpha)}$ and computes proof π_{PS} for language $\mathcal{L}_{\text{PS}}$ with witness (κ, α) , i.e., a proof that Ψ is a valid signature under (κ, α) and that $m = g_2^{1/(\kappa+\alpha)}$. We will discuss the algorithm for the validation of this blind signature next.

Verification. The verification algorithm consists of verifying the proofs π_{PS} .

$\text{Verify}(\text{pk}, m, \sigma := \pi_{\text{PS}})$

1: **return** $\Pi.\text{Verify}(\text{crs}, x := (\text{pk}, m), \pi_{\text{PS}})$

4.1 Security

In order to prove the security of our construction, we are required to show that it is one-more unforgeable, and recipient and context blind. For the former, we will make use of the following non-interactive variant of the original PS assumption (Definition 2.6). Notably, unlike the original assumption, our assumption variant is naturally *falsifiable*.

Definition 4.1 (q -type PS Problem (qPS)). Let $(\mathbb{G}, \tilde{\mathbb{G}})$ be pairing friendly groups of order p with a bilinear pairing $e: \mathbb{G} \times \tilde{\mathbb{G}} \rightarrow \tilde{\mathbb{G}}$. For $x, y, \mu_1, \dots, \mu_q \leftarrow \mathbb{Z}_p$, and $h_1, \dots, h_q \leftarrow \mathbb{G}$, the q -type PS problem asks that given $(p, g, \tilde{g}, e, \tilde{X} := \tilde{g}^x, \tilde{Y} := \tilde{g}^y, \{\mu_i, h_i, h_i^{x+y\mu_i}\}_{i \in [q]})$ find a $(\mu, h, h^{x+y\mu})$ triple such that $\mu \neq \mu_i$ for all $i \in [q]$ and $h \neq 1_{\mathbb{G}}$.

Theorem 4.2 (One-more unforgeability). *Assuming the qPS problem is hard and Π_R and Π_{PS} online knowledge extractable, Construction 1 is one-more unforgeable (in the random oracle model).*

Proof. We prove this theorem via a reduction to the hardness of the q -type PS assumption. Formally, suppose a PPT adversary $\mathcal{A}$ has non negligible advantage against one-more unforgeability of the NIBS scheme; we will then build a reduction $\mathcal{B}$ that will break the q -type PS assumption with non-negligible advantage as follows:

- Given $(p, g, \tilde{g}, e, \tilde{X} := \tilde{g}^x, \tilde{Y} := \tilde{g}^y, \{\mu_i, h_i, H_i := h_i^{x+y\mu_i}\}_{i \in [q]})$ from the qPS challenger C , the reduction $\mathcal{B}$ samples $\beta_1, \beta_2, z \leftarrow \mathbb{Z}_p^*$ and sends $\text{pp} := (p, g, \tilde{g}, e, g_1 := g^{\beta_1}, g_2 := g^{\beta_2}, \text{crs}_R, \text{crs}_{PS})$ and $\text{pk} := (\tilde{X}, \tilde{Y}_1 := \tilde{Y}, \tilde{Y}_2 := \tilde{Y}^z)$ to the adversary $\mathcal{A}$. Here crs_R and crs_{PS} are the honestly generated reference strings for languages $\mathcal{L}_R$ and $\mathcal{L}_{PS}$ respectively. $\mathcal{B}$ internally initializes $\mathcal{I}, \mathcal{K}, Q, := \emptyset$.
- For any O_{ISSUE} query of the form (pk_R, c) by $\mathcal{A}$, the reduction $\mathcal{B}$ responds as follows:
 - It checks if ℓ has exceeded q . If so, the reduction aborts and continues otherwise.
 - It first checks if $(\text{pk}_R, \alpha) \in \mathcal{K}$. Otherwise, after parsing pk_R as (pk'_R, π_R) it runs the extractor $\mathcal{E}_R$ on $(x := (g_1, \text{pk}'_R), \pi_R)$ to obtain the witness α satisfying $\text{pk}'_R = g_1^{1/\alpha}$. If not already in the set, it also updates $\mathcal{K} \leftarrow \mathcal{K} \cup \{(\text{pk}'_R, \alpha)\}$.
 - If for any $j \in [q]$, $(\text{pk}_R, c, j) \in Q$ it sets $i = j$; otherwise it sample $i \leftarrow [q] \setminus \mathcal{I}$ and, in both cases, sets $h := h_i$ and $\kappa := \mu_i - z\alpha$. It also internally updates $Q \leftarrow Q \cup \{(\text{pk}_R, c, i)\}$ and $\mathcal{I} \leftarrow \mathcal{I} \cup \{i\}$.
 - It then samples $r \leftarrow \mathbb{Z}_p$ and sends $\tilde{\sigma} := (A := H_i^{1+\alpha r}, B := H_i^{-r})$ to $\mathcal{A}$.
- For any $H_{\mathbb{G}}$ query of the form (pk_R, c) from $\mathcal{A}$, $\mathcal{B}$ responds as follows:
 - It checks if $[q] \setminus \mathcal{I} = \emptyset$. If so, it outputs $h_i := g^\gamma$ for $\gamma \leftarrow \mathbb{Z}_p$ and stores (pk_R, c, γ) . It continues otherwise.
 - If for any $j \in [q]$, $(\text{pk}_R, c, j) \in Q$; if so, it sets $i = j$. Otherwise it sample $i \leftarrow [q] \setminus \mathcal{I}$ and sends h_i to $\mathcal{A}$. It also internally updates $Q \leftarrow Q \cup \{(\text{pk}_R, c, i)\}$ and $\mathcal{I} \leftarrow \mathcal{I} \cup \{i\}$.
- For any $H_{\mathbb{Z}}$ query of the form (pk_R, c) from $\mathcal{A}$, $\mathcal{B}$ responds as follows:
 - It checks if $[q] \setminus \mathcal{I} = \emptyset$. If so, it outputs $\kappa_i := g^\gamma$ for $\gamma \leftarrow \mathbb{Z}_p$ and stores (pk_R, c, γ) . It continues otherwise.
 - It first checks if $(\text{pk}_R, \alpha) \in \mathcal{K}$. Otherwise, it parses pk_R as (pk'_R, π_R) and then runs the extractor $\mathcal{E}_R$ on $(x := (g_1, \text{pk}'_R), \pi_R)$ to obtain the witness α satisfying $\text{pk}'_R = g_1^{1/\alpha}$. If not already in the set, it also updates $\mathcal{K} \leftarrow \mathcal{K} \cup \{(\text{pk}'_R, \alpha)\}$.
 - If for any $j \in [q]$, $(\text{pk}_R, c, j) \in Q$ it sets $i = j$; otherwise it sample $i \leftarrow [q] \setminus \mathcal{I}$ and, in both cases, outputs $\mu_i - z\alpha$. It also internally updates $Q \leftarrow Q \cup \{(\text{pk}_R, c, i)\}$ and $\mathcal{I} \leftarrow \mathcal{I} \cup \{i\}$.
- Finally, after making $\ell \leq q = \text{poly}(\lambda)$ queries, when $\mathcal{A}$ outputs its (one-more) forgery $\{m_i, \sigma_i\}_{i \in [\ell+1]}$, $\mathcal{B}$ does the following:

- For every $i \in [\ell + 1]$, it runs the extractor $\mathcal{E}_{\text{PS}}$ on $(x := (\tilde{X}, \tilde{Y}_1, \tilde{Y}_2, m_i, \Psi_i), \pi_{\text{PS}}^{(i)})$ to obtain the witness (κ_i, α_i) satisfying $\mathcal{L}_{\text{PS}}$.⁹
- If not all $(\kappa_i + z\alpha_i)$ are distinct, then $\mathcal{B}$ aborts. Otherwise, since $\mathcal{A}$ made exactly ℓ queries to $\mathcal{O}_{\text{Issue}}$, it follows by a simple pigeonhole argument that there exists at least one $i^* \in [\ell + 1]$ such that the corresponding $(\kappa_{i^*} + z\alpha_{i^*})$ does not equal any of the messages μ_i 's with respect to any of the ℓ issuances. If for every such i^* , $\mathcal{B}$ returned κ_{i^*} as the response to a $\text{H}_{\mathbb{Z}}$ query, it aborts. Otherwise, it picks any of the remaining (unqueried) i^* , and sends the corresponding $(\mu^* := \kappa_{i^*} + z\alpha_{i^*}, \Psi_{i^*})$ to $\mathcal{C}$.

Since, by construction, μ^* is not in the qPS challenge, and Ψ_{i^*} is a valid PS signature on it, $\mathcal{B}$ wins with probability

$$\text{Adv}_{\mathcal{B}}^{\text{qPS}}(\lambda) \leq \text{Adv}_{\mathcal{A}}^{\text{OM-Unf}}(\lambda) + \text{Adv}_{\mathcal{A}}^{\text{KSnd,R}}(\lambda) + \text{Adv}_{\mathcal{A}}^{\text{KSnd,PS}}(\lambda) + \frac{q}{p} + \Pr[\text{abort}],$$

where q/p is the upper bound on the probability that the chosen κ_{i^*} corresponds to one of $(q - |\mathcal{I}|)$ qPS messages chosen uniformly from $\mathbb{Z}_p$. Also, by assumption, $\text{Adv}_{\mathcal{A}}^{\text{KSnd,R}}(\lambda)$ and $\text{Adv}_{\mathcal{A}}^{\text{KSnd,PS}}(\lambda)$ are negligible, so to complete the proof it only remains to be shown that the probability that $\mathcal{B}$ aborts is also negligible. In particular, we analyze two abort conditions—

- (i) *Not all $(\kappa_i + z\alpha_i)$ are distinct:* in other words, there exists some $i, j \in [\ell + 1]$ such that $\kappa_i + z\alpha_i = \kappa_j + z\alpha_j$. However, since a valid one-more forgery requires that all messages defined as $m_i := g_2^{1/(\kappa_i + \alpha_i)}$ be distinct, then we also have that $\kappa_i + \alpha_i \neq \kappa_j + \alpha_j$. Notice that since $z \in \mathbb{Z}_p^*$, $\kappa_i + z\alpha_i = \kappa_j + z\alpha_j$ implies that $\kappa_i \neq \kappa_j \Leftrightarrow \alpha_i \neq \alpha_j$ and since $\kappa_i + \alpha_i \neq \kappa_j + \alpha_j$, we get that $\kappa_i \neq \kappa_j$ and $\alpha_i \neq \alpha_j$. However, given such a pair (κ_i, α_i) and (κ_j, α_j) , $\mathcal{A}$ could easily compute $z := \frac{\kappa_j - \kappa_i}{\alpha_i - \alpha_j} \pmod{p}$ which would break the discrete log base $\tilde{Y}$.
- (ii) *$\mathcal{B}$ returned κ_{i^*} as the response to a $\text{H}_{\mathbb{Z}}$ query:* recall that any such κ_{i^*} does not have a corresponding issuance by $\mathcal{O}_{\text{Issue}}$. Then, we can bound the probability of this event as follows. Let q_{H} be the parameterized bound on the number of hash queries of any type that $\mathcal{A}$ can make. Then, $\mathcal{B}$ uniformly chooses one $\text{H}_{\mathbb{Z}}$ query and instead of answering it using a qPS message when $[q] \setminus \mathcal{I} \neq \emptyset$, it sends a freshly sampled uniform value $\rho \leftarrow \mathbb{Z}_p$. Therefore, with probability, at most $(\ell + 1)/q_{\text{H}}$, κ_{i^*} is the uniform value, ρ , for which $\mathcal{A}$ does not have a valid issuance but is able to produce a valid PS signature. Consequently, the probability of this event at most $((\ell + 1)/q) \cdot \text{Adv}_{\mathcal{A}}^{\text{qPS}}(\lambda)$.

Therefore,

$$\Pr[\text{abort}] \leq \frac{\ell + 1}{q_{\text{H}}} \cdot \text{Adv}_{\mathcal{A}}^{\text{qPS}}(\lambda) + \text{Adv}_{\mathcal{A}}^{\text{DLOG}}(\lambda).$$

□

⁹A slightly tighter reduction is likely possible here: instead of extracting all $(\ell + 1)$ witnesses, $\mathcal{B}$ computes all possible $g_2^{1/(\kappa + \alpha)}$ values (note it extracts all α) and for every such valid (κ, α) tuple appearing as a message m_i it checks whether there was a corresponding issuance. Notably, there must be at least one such tuple without an issuance. It runs $\mathcal{E}_{\text{PS}}$ on the proof π_{PS} of any such forgery.

Remark 4.3 (Honest key assumption.). To prove blindness we will require the honest key model [MOR01, Bol03]. This model requires the attacker to also provide the secret key sk corresponding to the public key pk in the blindness experiments. We remark that a NI(T)BS scheme that is secure in the honest key model can be upgraded into a fully secure scheme simply by having the signer provide a NIZK argument of knowledge of the secret key sk as part of pk . In the security proof, the reduction can then extract sk from the NIZKAoK and run the reduction for blindness in the honest key model.

Theorem 4.4 (Recipient blindness). *Assuming the sDDH and qDDH problems are hard in $\mathbb{G}$ and proof systems Π_R and Π_{PS} are zero-knowledge, Construction 1 is recipient blind (in the random oracle and honest key models).*

Proof. We proceed through a series of hybrids, where we use $\text{Adv}_{\mathcal{A}}^j$ to denote the advantage of a PPT adversary $\mathcal{A}$ in hybrid $\mathbf{H}_j$.

- Hybrid $\mathbf{H}_0$: This is the original strong recipient blindness experiment defined with respect to Construction 1.
- Hybrid $\mathbf{H}_1$: This is the same as $\mathbf{H}_0$ except, we simulate the proofs $\pi_R^{(0)}$ and $\pi_R^{(1)}$ given to the adversary as part of the recipients' public keys. Then assuming that the proof system for language $\mathcal{L}_R$ satisfies zero-knowledge, we have

$$|\text{Adv}_{\mathcal{A}}^0 - \text{Adv}_{\mathcal{A}}^1| \leq 2 \cdot \text{Adv}_{\mathcal{B}}^{\text{ZK}}(\lambda).$$

Where the factor 2 follows from the use simple sub-hybrids where first apply the change to R_0 and then to R_1 . It follows that hybrids $\mathbf{H}_0$ and $\mathbf{H}_1$ are indistinguishable to the adversary.

- Hybrid $\mathbf{H}_2$: This is the same as $\mathbf{H}_1$ except, on every Obtain query $(b, pk, \bar{\sigma}, c)$ it runs all the steps of Obtain($sk_{R_b}, pk, c, \bar{\sigma}$) but instead of computing the proof π_{PS} honestly, it uses simulator $\mathcal{S}(\text{crs}_{PS}, x)$ with $x = (pk, m)$.

Assuming the proof system for language $\mathcal{L}_{PS}$ is zero-knowledge, it follows that hybrids $\mathbf{H}_1$ and $\mathbf{H}_2$ are indistinguishable to the adversary. Let q_O be an upper bound on the number of Obtain queries the adversary makes. Then we have

$$|\text{Adv}_{\mathcal{A}}^2 - \text{Adv}_{\mathcal{A}}^1| \leq q_O \cdot \text{Adv}_{\mathcal{B}}^{\text{ZK}}(\lambda).$$

The above claim follows by using simple sub-hybrids, where we replace the proof in each query separately.

- Hybrid $\mathbf{H}_3$: This is the same as $\mathbf{H}_2$ except, we also simulate the two challenge proofs $\pi_{PS}^{(0)}$ and $\pi_{PS}^{(1)}$ given to the adversary. It follows that,

$$|\text{Adv}_{\mathcal{A}}^3 - \text{Adv}_{\mathcal{A}}^2| \leq 2 \cdot \text{Adv}_{\mathcal{B}}^{\text{ZK}}(\lambda).$$

- **Hybrid $\mathbf{H}_4$:** This is the same as $\mathbf{H}_3$ except, we now make the PS signature independent of the message (κ, α) . Specifically, we replace each ψ_2 component of every signature with a uniform $\phi \leftarrow \$ \mathbb{G}$. We argue that $\mathbf{H}_4$ is indistinguishable from $\mathbf{H}_3$ if the sDDH assumption holds in $\mathbb{G}$. More formally, we have that

$$|\text{Adv}_{\mathcal{A}}^4 - \text{Adv}_{\mathcal{A}}^3| \leq \text{Adv}_{\mathcal{B}}^{\text{sDDH}}(\lambda) + \Pr[\text{abort}] .$$

We further break this down into two sub-hybrids where we first apply the change to R_0 's signatures and then to R_1 's signatures. Then, in the first sub-hybrid, the reduction algorithm $\mathcal{B}$ instantiates a sDDH challenger over $\mathbb{G}$ and receives the corresponding challenge

$$\mathcal{D}_b^{\text{sDDH}} := (p, g_1, g_1^\alpha, g_1^{1/\alpha}, g_1^\beta, T) .$$

It samples a generator g with a known discrete log δ base g_1 , and uses g and the group $\mathbb{G}$ to instantiate the public parameters in the setup. It then sets the first element of pk_{R_0} to $g_1^{1/\alpha}$ it received from the sDDH challenger and the rest of the values as it did in the previous hybrid.

Now, on the i^{th} $H_{\mathbb{G}}$ query with input (pk_{R_0}, c_i) , $\mathcal{B}$ first looks up whether the tuple has been queried before, and if so returns the corresponding entry; otherwise it programs the random oracle by sampling $v_i \leftarrow \$ \mathbb{Z}_p^*$ and returns $h_i := g_1^{v_i}$. It also stores $(\text{pk}_{R_0}, c_i, v_i)$. For all other types of $H_{\mathbb{G}}$ queries, it answers them as before. It also answers any fresh $H_{\mathbb{Z}}$ queries by sampling a uniform $\kappa_i \leftarrow \$ \mathbb{Z}_p$ and storing the corresponding $(\text{pk}_{R_0}, c_i, \kappa_i)$ to answer any repeated queries.

For the k^{th} admissible Obtain query of the form $(0, \text{pk}, c_k, \bar{\sigma}_k)$ it looks up $(\text{pk}_{R_0}, c_k, \cdot)$ in its $H_{\mathbb{G}}$ queries and if no such entry is found, it aborts. Similarly, if no corresponding entry is found in its $H_{\mathbb{Z}}$ queries it also aborts. Otherwise, $\mathcal{B}$ uses the corresponding entries $v_{(k)}, \kappa_{(k)}$ along with knowledge of the adversary's secret key (x, y_1, y_2) to compute the PS signature as $((g_1^\beta)^{\delta v_{(k)}}, (g_1^\beta)^{\delta v_{(k)} \cdot (x + y_1 \kappa_{(k)})} \cdot T^{\delta v_{(k)} \cdot y_2})$. All other Obtain queries are answered as before. $\mathcal{B}$ similarly computes the final challenge PS signature Ψ_0 .

Observe that for any PS signature corresponding to pk_{R_0} we have that $\psi_1 = (g_1^\beta)^{\delta v_{(k)}} = h_{(k)}^\beta$ and moreover, if $T = g_1^{\alpha\beta}$ then $\psi_2 = (g_1^\beta)^{\delta v_{(k)} \cdot (x + y_1 \kappa_{(k)})} \cdot T^{\delta v_{(k)} \cdot y_2} = h_{(k)}^{\beta \cdot (x + y_1 \kappa_{(k)})} \cdot (g_1^{\alpha\beta})^{\delta v_{(k)} \cdot y_2} = h_{(k)}^{\beta \cdot (x + y_1 \kappa_{(k)})} \cdot h_{(k)}^{\beta\alpha y_2} = h_{(k)}^{\beta \cdot (x + y_1 \kappa_{(k)} + y_2 \alpha)}$. On the other hand, if T is a uniformly sampled element of $\mathbb{G}$, then ψ_2 is indistinguishable from uniform. In other words, in case of hybrid $\mathbf{H}_3$ we must have $\mathcal{D}_b^{\text{sDDH}} = \mathcal{D}_0^{\text{sDDH}}$, and in hybrid $\mathbf{H}_4$, $\mathcal{D}_b^{\text{sDDH}} = \mathcal{D}_1^{\text{sDDH}}$ as desired. The claim follows by repeating this reduction for R_1 .

- **Hybrid $\mathbf{H}_5$:** This is the same as $\mathbf{H}_4$ except, with a minor change. Specifically, let (m_b, σ_b) and (m_{1-b}, σ_{1-b}) be the message-signature pairs given to the adversary. Instead of computing messages m_b according to the construction, it is chosen at random from $\mathbb{G}$, i.e., $m_b \leftarrow \$ \mathbb{G}$. We argue that $\mathbf{H}_5$ is indistinguishable from $\mathbf{H}_4$ if the qDDH assumption holds in $\mathbb{G}$. More formally, we have that

$$|\text{Adv}_{\mathcal{A}}^5 - \text{Adv}_{\mathcal{A}}^4| \leq 1/(q+1) \cdot \text{Adv}_{\mathcal{B}}^{\text{qDDH}}(\lambda) + \Pr[\text{abort}] ,$$

where the adversary is only allowed to make at most $(q+1)$ queries to the $H_{\mathbb{Z}}$ oracle.

In particular, the reduction algorithm $\mathcal{B}$ instantiates a qDDHI challenger over $\mathbb{G}$ and receives the corresponding challenge

$$\mathcal{D}_b^{\text{qDDHI}} := (p, g, g^\alpha, g^{\alpha^2}, \dots, g^{\alpha^q}, T).$$

It uses g and the group $\mathbb{G}$ to instantiate the public parameters in the setup. In practice, g_1 and g_2 will be generated using the random oracle via hashing to a group. So, $\mathcal{B}$ can program the oracle to output predetermined by $\mathcal{B}$ values for g_1 and g_2 . Specifically, $\mathcal{B}$ randomly picks a value $v \leftarrow \mathbb{Z}_p^*$ and sets $g_1 = (g^\alpha)^v$ and sets the first element of pk_{R_b} to $g_1^{1/\alpha_b} = g^v$. Thus, $\text{sk}_{R_b} = \alpha_b = \alpha$, but since $\mathcal{B}$ received α as part of the challenge, it does not know sk_{R_b} . However, since all the proofs are simulated, we only need to show how to generate messages for this recipient.

In the next step, $\mathcal{B}$ picks uniformly random values $\kappa'_1, \dots, \kappa'_{q+1}$ from $\mathbb{Z}_p^*$ and programs the oracle H_Z on the i -th query output κ'_i . It then picks one of the κ_b at random and aborts if κ_b is not the output of query $H_Z(\text{pk}_{R_b}, c_b)$. Note that $\mathcal{B}$ continues the simulation with probability $1/(q+1)$.

It then defines the polynomial $f(X) := \left(\prod_{i=1}^{q+1} (X + \kappa'_i)\right) (X + \kappa_b)^{-1}$ and sets $g_2 = g^{f(\alpha)}$. Note that since f is of degree q , $\mathcal{B}$ can use $\mathcal{D}_b^{\text{qDDHI}}$ to compute g_2 . It is easy to see now that $\mathcal{B}$ can simulate all Obtain oracle queries of the adversary for the recipient with secret key sk_{R_b} . Given an adversary's Obtain query of the form $(b, \text{pk}, \tilde{\sigma}, c)$, $\mathcal{B}$ first looks up $\kappa_j = H_Z(\text{pk}_{R_b}, c)$ and then defines the polynomial $f_j(X) := f(X)/(X + \kappa'_j)$. Note that because of the way we defined f , it is divisible by $(X + \kappa'_j)$. Using the values from $\mathcal{D}_b^{\text{qDDHI}}$, $\mathcal{B}$ computes the output of the Obtain query (m, σ) as $m := g^{f_j(\alpha)}$, while it also uses the simulator for the proof system to generate $\sigma := \pi_{\text{PS}}$ (as per prior hybrids).

Finally, $\mathcal{B}$ prepares (m_b, σ_b) . For σ_b is uses the simulator, while m_b is computed as follows. By definition m_b should be $g_2^{1/(\alpha + \kappa_b)}$. However, since we set $g_2 = g^{f(\alpha)}$ and $(X + \kappa_b)$ is not a factor of the polynomial $f(X)$, we can rewrite message m_b as $g^{\prod_{i=1}^{q+1} a_i \cdot \alpha^i}$ for coefficient a_i computable using the coefficient of the polynomial f and κ_b . What is more, because $(X + \kappa_b)$ is not a factor we must have $a_{-1} \neq 0$. For each $i \in [q]$, let us denote by $P_i = g^{\alpha^i}$ from $\mathcal{D}_b^{\text{qDDHI}}$.

Thus, $\mathcal{B}$ computes $m_b = T^{a_{-1}} \cdot \prod_{i=0}^{q-1} (P_i)^{a_i}$. It is easy to see that in case of hybrid $\mathbf{H}_4$, $\mathcal{D}_b^{\text{qDDHI}} = \mathcal{D}_0^{\text{qDDHI}}$, and in hybrid $\mathbf{H}_5$, $\mathcal{D}_b^{\text{qDDHI}} = \mathcal{D}_1^{\text{qDDHI}}$ since T is random. So the claim follows.

- Hybrid $\mathbf{H}_6$: This is the same as $\mathbf{H}_5$ except, but we apply the same changes now to (m_{1-b}, σ_{1-b}) . We argue that $\mathbf{H}_6$ is indistinguishable from $\mathbf{H}_5$ if the qDDHI assumption holds in $\mathbb{G}$. More formally, we have

$$|\text{Adv}_{\mathcal{A}}^6 - \text{Adv}_{\mathcal{A}}^5| \leq 1/(q+1) \cdot \text{Adv}_{\mathcal{B}}^{\text{qDDHI}}(\lambda) \Pr[\text{abort}],$$

where the adversary is only allowed to make at most $(q+1)$ queries to the H_Z oracle. This follows by using the same argument as above.

It is clear that the probability $\Pr[\text{abort}]$ in all of the above is $1/p$ as it corresponds to guessing the output of the random oracle. Lastly, we claim that $\text{Adv}_{\mathcal{A}}^6 = 1/2$. This follows directly from the way that the challenge message-signature pairs (m_b, σ_b) and (m_{1-b}, σ_{1-b}) are generated in $\mathbf{H}_6$. Specifically, (i) each $\pi_{\text{PS}}^{(b)}$ is a simulated zero-knowledge proofs computed without using the witness; (ii) each PS signature Ψ_b is also independent of the message (which is the witness); and (iii) finally the NIBS messages are

random elements in $\mathbb{G}$ and consequently independent of the witness elements. Therefore, the challenge message-signature pairs are completely independent of pk_{R_0} and pk_{R_1} , which completes the proof. $\square$

Theorem 4.5 (Context blindness). *Assuming the sDDH and qDDH problems are hard in $\mathbb{G}$ and proof systems Π_R and Π_{PS} are zero-knowledge, Construction 1 is context blind (in the random oracle and honest key models).*

Sketch. We only sketch the proof of context blindness since it follows the same steps as the proof of recipient blindness. In the first step, we simulate all the proofs inside the public keys $\text{pk}_R^{(b)}$. Then, we simulate all the proofs created as part of the Obtain oracle using the zero-knowledge property as well as the proofs from the challenge signatures. In the next step, we replace the PS signatures with random group elements under the sDDH assumption. We then use the qDDH assumption to show that an adversary cannot notice if we replace one of the two messages for the challenge message-signature pair with a random element from $\mathbb{G}$. Note that due to the qDDH assumption, we can still create all the messages for the Obtain queries according to the protocol. Moreover, any adversary able to distinguish this change immediately leads to an adversary against the qDDH assumption. The final step is to replace the second of the two messages with a random element from $\mathbb{G}$. With this change, the challenge to the adversary is composed of two random elements in $\mathbb{G}$ and simulated proofs independent of the context. Thus, the chances of the adversary are equivalent to random guessing, i.e., $1/2$. $\square$

4.2 The concrete NIZK proof system

We will now show how to efficiently instantiate the NIZK proof systems for languages $\mathcal{L}_R$ and $\mathcal{L}_{PS}$ and show that our instantiations satisfy the necessary properties.

Proof for language $\mathcal{L}_R$. Recipients can use a proof of knowledge of α , where $\text{pk}'_R := g_1^{1/\alpha}$. This standard proof of discrete logarithm knowledge can be instantiated as follows. Compute $T_R := (\text{pk}'_R)^{\alpha'}$, for some random $\alpha' \leftarrow \mathbb{Z}_p^*$, and challenge $c_R := H(g_1, \text{pk}'_R, T_R)$. Set proof $\pi_R = (c_R, s_R)$, where $s_R := \alpha' - c_R \cdot \alpha \pmod{p}$. To verify the proof, the verifier computes $T'_R = g_1^{c_R} \cdot (\text{pk}'_R)^{s_R}$ and returns 1 iff $H(g_1, \text{pk}'_R, T'_R) = c_R$.

Proof for language $\mathcal{L}_{PS}$. The proof for language $\mathcal{L}_{PS}$ is comprised of a proof of knowledge of (κ, α) such that $\Psi = (\psi_1, \psi_2)$ is a valid PS-signature on (κ, α) and $m = g_2^{\kappa + \alpha}$. In order to instantiate this proof, we first recall that the verification check for a PS-signature Ψ on (κ, α) is

$$\underbrace{e(\psi_1, \tilde{Y}_1)}_{E_1} \cdot \underbrace{e(\psi_1, \tilde{Y}_2)}_{E_2} = \underbrace{e(\psi_1, \tilde{X}^{-1}) \cdot e(\psi_2, \tilde{g})}_{E_3}.$$

Now in order to compute this proof, the prover first samples $(\kappa', \alpha') \leftarrow (\mathbb{Z}_p^*)^2$, and then computes commitments $T_1 := E_1^{\kappa'} \cdot E_2^{\alpha'}$ and $T_2 := m^{\kappa' + \alpha'}$. Next, it computes the challenge $c_{PS} = H(g, g_2, \tilde{g}, \tilde{X}, \tilde{Y}_1, \tilde{Y}_2, \psi_1, \psi_2, E_1, E_2, E_3, T_1, T_2)$ and sends the proof $\pi_{PS} := (c_{PS}, s_\kappa, s_\alpha)$ where $s_\kappa := \kappa' - c_{PS} \cdot \kappa \pmod{p}$ (similarly, s_α). To verify the proof, the verifier first computes E_1, E_2 and E_3 as defined, then sets $T_1 := E_1^{s_\kappa} \cdot E_2^{s_\alpha} \cdot E_3^{-c_{PS}}$ and $T_2 := m^{s_\kappa + s_\alpha} \cdot g_2^{-c_{PS}}$ and finally outputs 1 iff $H(g, g_2, \tilde{g}, \tilde{X}, \tilde{Y}_1, \tilde{Y}_2, \psi_1, \psi_2, E_1, E_2, E_3, T_1, T_2) = c_{PS}$.

Summarizing all the above, the proof π_R consists of 2 elements (c_R, s_R) in $\mathbb{Z}_p^*$; and the proof π_{PS} consists of the randomized PS signature (ψ_1, ψ_2) and the proof $\pi_{PS} := (c_{PS}, s_\kappa, s_\alpha)$. Therefore, π_{PS} consists of 2 elements in $\mathbb{G}$ and 3 elements in $\mathbb{Z}_p^*$.

We will now show that given any such NIZK proof and the corresponding instance, our reduction in the proof of Theorem 4.2 can indeed extract the witness in an *online* manner. Our proof relies on the algebraic group model (AGM).

Lemma 4.6. *Proof system Π_R is online knowledge extractable in the algebraic group model.*

Proof. We give an extractor $\mathcal{E}_R$ that, given an instance $(\phi_1, \phi_2) \in \mathbb{G}^2$ and a proof π_R , is able to extract $\alpha \in \mathbb{Z}_p^*$ satisfying $\phi_2 = \phi_1^\alpha$. Observe that the adversary outputs π_R as part of its query to the $\mathcal{O}_{\text{Issue}}$ oracle, and that every such query has the form (pk_R, c) where $\text{pk}_R =: (\text{pk}'_R, \pi_R) \in \mathbb{G} \times \mathbb{Z}_p^* \times \mathbb{Z}_p^*$. In particular, for the k^{th} such query, the algebraic adversary must also output a representation $(a_0, a_1, a_2, \{a_{3,i}\}_{i \in [|Q|]}, \{a_{4,j}, a_{5,j}\}_{j \in [k-1]})$ such that

$$\text{pk}'_{R_k} = g^{a_0} \cdot g_1^{a_1} \cdot g_2^{a_2} \cdot \prod_{i \in [|Q|]} h_i^{a_{3,i}} \cdot \prod_{j \in [k-1]} (A_j^{a_{4,j}} \cdot B_j^{a_{5,j}}),$$

where Q is the set of all query and response pairs to $\mathcal{H}_{\mathbb{G}}$. Then, clearly for any such pk'_{R_k} , we have by soundness of the proof system that

$$g_1^{1/\alpha_k} = g^{a_0} \cdot g_1^{a_1} \cdot g_2^{a_2} \cdot \prod_{i \in [|Q|]} h_i^{a_{3,i}} \cdot \prod_{j \in [k-1]} (A_j^{a_{4,j}} \cdot B_j^{a_{5,j}}).$$

Expanding each $A_j = h_j^{x+y_1\kappa_j}$ and $B_j = \text{pk}'_{R_j} \cdot h_j^{y_2} = g_1^{1/\alpha_j} \cdot h_j^{y_2}$, (for indeterminates x, y_1 and y_2), we can re-write the right-hand side so that

$$g_1^{1/\alpha_k} = g^{a_0} \cdot g_1^{a_1 + \sum_{j \in [k-1]} (a_{4,j}/\alpha_j)} \cdot g_2^{a_2} \cdot \prod_{i \in [|Q|]} h_i^{a_{3,i}} \cdot \prod_{j \in [k-1]} h_j^{a_{4,j}x + a_{4,j}y_1\kappa_j + a_{5,j}y_2}. \quad (1)$$

Now, let each $h_i = g^{\gamma_i}$ for some indeterminate $\gamma_i \in \mathbb{Z}_p$. Then, passing to the discrete-log base g and re-arranging gives

$$\beta_1 \cdot \left(1/\alpha_k - a_1 - \sum_{j \in [k-1]} (a_{4,j}/\alpha_j) \right) - \beta_2 \cdot a_2 - \sum_{i \in [|Q|]} \gamma_i \cdot a_{3,i} + \sum_{j \in [k-1]} \gamma_j \cdot (a_{4,j}x + a_{4,j}y_1\kappa_j + a_{5,j}y_2\alpha) = a_0 \pmod{p}.$$

For uniform choices of β_1, β_2 and all γ_i 's, except with probability at most $1/p$, this equation holds only when

$$\forall i \in [|Q|], j \in [k-1] : a_2 = a_{3,i} = a_{4,j} = a_{5,j} = 0 \text{ and } 1/\alpha_k = a_1.$$

An additional caveat completes the argument. Specifically, the extractor may not control the actual values of the γ_i when the bases h_i are supplied by an external challenger. The argument above therefore only guarantees vanishing exponents against uniform γ_i . However, except with some negligible probability $\nu(\lambda)$, the h_i 's are computationally indistinguishable from uniform¹⁰. Combining the above, $\mathcal{E}_R$ fails to extract the witness for π_R with probability at most $\frac{1}{p} + \nu(\lambda) + \text{Adv}_{\mathcal{A}}^{\text{Snd}}(\lambda)$. $\square$

Lemma 4.7. *Proof system Π_{PS} is online knowledge extractable in the algebraic group model.*

¹⁰A similar argument was made in [ABM15] to prove simulation extractability of Schnorr proof system for password-authenticated key exchange, where the crux of the argument fell on showing that the challenge bases were indistinguishable from uniformly random bases.

Proof. We give an extractor $\mathcal{E}_{\text{PS}}$ that, given an instance $(m, \psi_1, \psi_2, \tilde{X}, \tilde{Y}_1, \tilde{Y}_2) \in \mathbb{G}^3 \times \tilde{\mathbb{G}}^3$ and a proof π_{PS} , is able to extract scalars $(\kappa, \alpha) \in (\mathbb{Z}_p^*)^2$ satisfying $e(\psi_1, \tilde{X}) \cdot e(\psi_1, \tilde{Y}_1)^\kappa \cdot e(\psi_1, \tilde{Y}_2)^\alpha = e(\psi_2, \tilde{g}_2)$ and $m = g_2^{1/(\kappa+\alpha)}$.

Recall that the adversary outputs π_{PS} as part of its (one-more) forgery, and that every such forgery consists of group elements (m, ψ_1, ψ_2) along with their representations in their respective bases. We can rewrite this representation similarly to (1) as

$$m = g^{a_0} \cdot g_1^{a_1 + \sum_{j \in [\ell]} (a_{4,j} / \alpha_j)} \cdot g_2^{a_2} \cdot \prod_{i \in [|Q|]} h_i^{a_{3,i}} \cdot \prod_{j \in [\ell]} h_j^{a_{4,j}x + a_{4,j}y_1\kappa_j + a_{5,j}y_2},$$

the honest relation for this commitment asserts that $m = g_2^{1/(\kappa+\alpha)}$ for some witness pair (κ, α) . We may thus equate the two group elements and rearrange to obtain

$$g^{a_0} \cdot g_1^{a_1 + \sum_{j \in [\ell]} (a_{4,j} / \alpha_j)} \cdot g_2^{a_2 - \frac{1}{\kappa+\alpha}} \cdot \prod_{i \in [|Q|]} h_i^{a_{3,i}} \cdot \prod_{j \in [\ell]} h_j^{a_{4,j}x + a_{4,j}y_1\kappa_j + a_{5,j}y_2} = 1.$$

We can show identically as for the case of π_{R} that with probability at most $\frac{1}{p} + v(\lambda) + \text{Adv}_{\mathcal{A}}^{\text{Snd}}(\lambda)$, the exponents other than a_2 are all zero, which allows $\mathcal{E}_{\text{PS}}$ to obtain

$$\kappa + \alpha = a_2^{-1}. \quad (2)$$

Next, given the representation $\psi_1 = g^{a_0} \cdot g_1^{a_1} \cdot g_2^{a_2} \cdot \prod_{i \in [|Q|]} h_i^{a_{3,i}} \cdot \prod_{j \in [\ell]} (A_j^{a_{4,j}} \cdot B_j^{a_{5,j}})$, we express it so that $\psi_1 = \prod_i R_i^{a'_i}$, where every R_i enumerates the individual $\mathbb{G}$ bases appearing in the system (i.e., $R_0 = g, R_1 = g_1, \dots$).

Observe that for any proof π_{PS} , there must exist $E_1 \in \hat{\mathbb{G}}$ such that

$$E_1 = e(\psi_1, \tilde{X} \cdot \tilde{Y}_1^\kappa \cdot \tilde{Y}_2^\alpha) = e(\psi_1, \tilde{g})^{x + y_1\kappa + y_2\alpha} = e(\psi_1, \tilde{g})^\tau,$$

where $\tau := x + y_1\kappa + y_2\alpha$. Moreover, with probability $1 - 1/p$, the algebraic adversary must have queried the random oracle for E_1 to generate the challenge c_{PS} . Thus, it must have also output a representation with respect to pairings over known group elements. Thus, we can write $E_1 = \prod_{i,j} e(S_i, \tilde{S}_j)^{b_{i,j}}$, where each S_i and $\tilde{S}_j$ themselves have known representations in their respective group bases. By bilinearity and the AGM-expansions of the $\mathbb{G}$ and $\tilde{\mathbb{G}}$ bases, we may re-write E_1 , by splitting and recombining the pairings so that each resulting individual pairing has a distinct R_i as the $\mathbb{G}$ term. Let $E_1 = \prod_{i,j} e(R_i, \tilde{R}_j)^{b'_{i,j}}$ be the resulting representation. Then we have that

$$\prod_{i,j} e(R_i, \tilde{R}_j)^{b'_{i,j}} = E_1 = e(\psi_1, \tilde{g})^\tau = e\left(\prod_i R_i^{a'_i}, \tilde{g}\right)^\tau = \prod_i e(R_i, \tilde{g})^{a'_i\tau}.$$

Rearranging,

$$\prod_{i,j} e(R_i, \tilde{R}_j)^{c_{i,j}} = 1 \pmod{p}, \quad (3)$$

where $c_{i,j} = b'_{i,j} - a'_i\tau \cdot \mathbf{1}_{\{j=k\}}$. Now, let $\hat{g} = e(g, \tilde{g})$, then by bilinearity, we have for all i, j that

$$e(R_i, \tilde{R}_j) = e(g, \tilde{g}_0)^{\beta_i\gamma_j} = \hat{g}^{\beta_i\gamma_j},$$

for some indeterminate $\beta_i, \gamma_j \in \mathbb{Z}_p$. Passing Equation (3) to the discrete-log base $\hat{g}$ under this substitution then gives

$$\sum_{i,j} \beta_i \gamma_j c_{i,j} = \sum_i \beta_i \left(\sum_j c_{i,j} \gamma_j \right) = 0.$$

Viewing the above as a bilinear polynomial in indeterminates $(\beta_i)_i$ and $(\gamma_j)_j$, we observe that one of the following two cases must be true:

- If the coefficient $c_{i,j}$ equals zero for all i, j , then necessarily $b'_{i,j} = a'_i \tau$ when $j = k$ and $b'_{i,j} = 0$ for $j \neq k$, and extraction succeeds deterministically by picking any i with $a'_i \neq 0$ and computing

$$\tau = b'_{i,k} / a'_i. \quad (4)$$

In this case, solving (2) and (4) gives us (κ, α) .

- Otherwise we have a nonzero bilinear polynomial and by a simple application of the Schwartz–Zippel lemma, we get that the probability (over uniformly random choices of the indeterminates β_i and γ_j) that this nonzero polynomial evaluates to zero is at most $2/p$.

Therefore, in the latter failure case the equality $E_1 = e(\psi_1, \tilde{g})^\tau$ can hold only with probability at most $2/p$ over the uniform choice of $(\beta_i)_i$ and $(\gamma_j)_j$. Combining all of the above, we get that the probability that the extractor fails to extract is at most $\frac{1}{p} + \frac{2}{p} + \nu(\lambda) + \text{Adv}_{\mathcal{A}}^{\text{Snd}}(\lambda)$. $\square$

We also note that the zero-knowledge properties of Π_R and Π_{PS} follow directly from that of the Schnorr proof system.

5 NITBS from PS Signatures

This section introduces our main result, a non-interactive threshold blind signatures construction from PS signatures. Our starting point is the non-interactive blind signature scheme presented in Section 4. Recall that the presignature was (A, B) where $A = h^{x+y_1\kappa} \cdot g_1^r$ and $B = (\text{pk}_R)^{-r} \cdot h^{y_2}$, where x, y_1 and y_2 are the secret keys of the signer. To construct a threshold version, we first use the standard t out of n secret sharing of those keys $([x]_i, [y_1]_i, [y_2]_i) \leftarrow \text{Share}_{t,n}((x, y_1, y_2))$. Equipped with one share of each key, a signer can create the presignature $A_i = h^{[x]_i + [y_1]_i \kappa} \cdot g_1^r$ and $B_i = (\text{pk}_R)^{-r} \cdot h^{[y_2]_i}$, and once combined to partial PS signatures $h^{[x]_i + [y_1]_i \kappa + [y_2]_i \alpha}$ we can use standard Lagrange coefficients Λ_i to reconstruct $h^{x+y_1\kappa+y_2\alpha}$. The remaining steps are similar to the scheme in Section 4. We give the details below.

Construction 2. Our construction requires hash functions $H_{\mathbb{G}} : \{0, 1\}^\lambda \rightarrow \mathbb{G}$ and $H_{\mathbb{Z}} : \{0, 1\}^\lambda \rightarrow \mathbb{Z}_p^*$. We will also make use of three non-interactive proof systems $\Pi = (\Pi.\text{Setup}, \Pi.\text{Prove}, \Pi.\text{Verify})$, for languages $\mathcal{L}_R$, $\mathcal{L}_{PS}$ as previously defined. Below, we describe only the main changes from the non-threshold construction (1).

Key generation. No change occurs in the recipient's key generation algorithm from Construction 1, and it is therefore omitted. The signer's key generation algorithm is also similar, except that except the secrets x, y_1, y_2 are t -out-of- n threshold secret shared among all n signers.

KeyGen_S(pp, t, n)

- 1: $(x, y_1, y_2) \leftarrow (\mathbb{Z}_p^*)^3$
- 2: $\tilde{X} := \tilde{g}^x, \tilde{Y}_1 := \tilde{g}^{y_1}, \tilde{Y}_2 := \tilde{g}^{y_2}$
- 3: $([x]_i, [y_1]_i, [y_2]_i) \leftarrow \text{Share}_{t,n}((x, y_1, y_2))$
- 4: **return** $\text{mpk} := (\tilde{X}, \tilde{Y}_1, \tilde{Y}_2),$
 $\{\text{sk}_i := ([x]_i, [y_1]_i, [y_2]_i)\}_{i \in [n]}$

Signer^(pk_R, c)

$\underbrace{[x]_i, [y_1]_i, [y_2]_i \in (\mathbb{Z}_p^*)^3}_{\text{sk}_i},$

..... Issue

if $\Pi.\text{Verify}(\text{crs}_R, x := (g_1, \text{pk}'_R), \pi_R) \neq 1$

then return $\perp$

$h := H_G(\text{pk}_R, c), \kappa := H_Z(\text{pk}_R, c)$

$r \leftarrow \mathbb{Z}_p^*, A := h^{[x]_i + [y_1]_i \cdot \kappa} \cdot g_1^r$

$B := (\text{pk}'_R)^{-r} \cdot h^{[y_2]_i}$

return $\bar{\sigma} := (A, B)$

..... Obtain

$\xrightarrow{\mathcal{S}, \{\bar{\sigma}_i := (A_i, B_i)\}_{i \in \mathcal{S}}, c}$

$u \leftarrow \mathbb{Z}_p^*, h := H_G(\text{pk}_R, c), \kappa := H_Z(\text{pk}_R, c)$

$m := g_2^{1/(\kappa + \alpha)}, \Psi := \left(h^u, \left(\prod_{i \in \mathcal{S}} (A_i \cdot B_i^{\alpha})^{\Lambda_i} \right)^u \right)$

$\pi_{PS} \leftarrow \Pi.\text{Prove} \left(\begin{array}{l} \text{crs}_{PS}, x := (\text{pk}, m) \\ w := (\Psi, \kappa, \alpha) \end{array} \right)$

return $(m, \sigma := \pi_{PS})$

Figure 5: Presignature issuance and blind signature generation for construction 2.

Presignature issuance and signature generation. The issuance algorithm is the same as in the case of the non-threshold construction, except that issuers are using their shares $([x]_i, [y_1]_i, [y_2]_i)$ for signing. However, the obtain algorithm differs slightly. First, given a set of signatures $(\bar{\sigma}_i = (A_i, B_i))_{i \in \mathcal{S}}$ for some subset $\mathcal{S} \subseteq [n]$ of signers, the recipient recomputes the generator $h := H_G(\text{pk}_R, c)$ and the partial message $\kappa := H_Z(\text{pk}_R, c)$ as before. Next, using the Lagrange coefficients Λ_i for each issuer in $\mathcal{S}$ (which it can pre-compute), it sets the PS signature Ψ as $(h^\rho, \prod_{i \in \mathcal{S}} (A_i \cdot B_i^\alpha)^{\rho \Lambda_i})$ for $\rho \leftarrow \mathbb{Z}_p$. The rest of the protocol is identical with the final blind signature being the PS signature Ψ on (κ, α) and the proof π_{PS} for $\mathcal{L}_{PS}$.

5.1 Security

Here, we prove the above construction's one-more unforgeability, recipient, and context blindness according to the framework in Section 3. In particular, we show the former property under the hardness of the (t, n) -threshold PS problem which, we in turn show, can be reduced to the one-more co-discrete logarithm assumption in the AGM. Recipient and context blindness, as we argue later, hold similarly to the non-threshold version under zero-knowledge of the proof systems and the hardness of the qDDHI problem.

Definition 5.1 ((t, n) -threshold PS Problem). Let $(\mathbb{G}, \tilde{\mathbb{G}})$ be pairing friendly groups of order p with a bilinear pairing $e : \mathbb{G} \times \tilde{\mathbb{G}} \rightarrow \hat{\mathbb{G}}$. For a uniformly random chosen polynomials $f_x(X)$ and $f_y(X)$ of degree t over $\mathbb{Z}_p[X]$, the (t, n) -threshold PS problem asks that given access to the oracles $O_1, O_2, O_{\text{Corrupt}}$ and $(p, g, \tilde{g}, e, \tilde{X} := \tilde{g}^{f_x(0)}, \tilde{Y} := \tilde{g}^{f_y(0)}, \{\tilde{X}_i := \tilde{g}^{f_x(i)}, \tilde{Y}_i := \tilde{g}^{f_y(i)}\}_{i \in [n]})$, find a $(\mu^*, h^*, (h^*)^{f_x(0)+f_y(0)\mu^*})$ triple such that $|\mathcal{C}| + |\mathcal{S}_{\mu^*}| \leq (t-1)$ and $h \neq 1_{\mathbb{G}}$. On input $\mu \in \mathbb{Z}_p$ oracle O_1 outputs $h \in \mathbb{G}$. On input of the same $\mu \in \mathbb{Z}_p$ and $i \in [n] \setminus \mathcal{C}$, oracle O_2 outputs $h^{f_x(i)+f_y(i)\mu}$. On any repeated queries, both oracles replay the same response as before and additionally, oracle O_2 adds i to a set $\mathcal{S}_{\mu}$. Finally, the corruption oracle O_{Corrupt} on input index $i \in [n]$, adds i to set $\mathcal{C}$ and returns $(f_x(i), f_y(i))$.

Lemma 5.2. *In the algebraic group model, the (t, n) -threshold PS assumption (Definition 5.1) holds under the one-more version of the co-discrete logarithm assumption.*

Proof. The idea for the proof is straightforward, and in particular, the same steps as in the reduction for the standard PS assumption (Lemma 2.7) can also be used here for embedding the discrete logarithm problem into one of the bases h_i . The critical difference is in how the reduction handles the computation of the discrete logarithm if embedded into the public key part $\tilde{X}$ and $\tilde{Y}$. In such a case, the one-more co-discrete logarithm is needed to allow adaptive corruptions.

The intuition here is that after $(t-1)$ queries to the corruption oracle, which can be modeled using the oracle O_{dlog} , the problem “downgrades” to a single unknown value on the side of the reduction and any adversary against the above assumption will allow the reduction to obtain this value and consequently compute all the required discrete logarithms for the one-more co-discrete logarithm assumption. We will show these particular steps by constructing such a reduction $\mathcal{B}$ for the case of embedding the secret into $\tilde{Y}$. The case for $\tilde{X}$ is analogous.

In the first step, the reduction $\mathcal{B}$ asks its challenger $\mathcal{C}$ in the one more co-discrete logarithm assumption for t tuples $(T_i, \tilde{T}_i) = (g^{t_i}, \tilde{g}^{t_i})$ for $i \in [0, t-1]$. In the next step, it picks a random polynomial $f_x(X)$ of degree $(t-1)$ and prepares $I = (p, g, \tilde{g}, e, \tilde{X} := \tilde{g}^{f_x(0)}, \tilde{Y} := \tilde{g}^{f_y(0)}, \{\tilde{X}_i := \tilde{g}^{f_x(i)}, \tilde{Y}_i := \tilde{g}^{f_y(i)}\}_{i \in [n]})$, where it sets $\tilde{Y}_i = \tilde{T}_i$ for all $i \in [t-1]$ and $\tilde{Y} = \tilde{T}_0$. Thus, by defining the polynomial $f_y(X)$ using fixed points $f_y(i) = t_i$ for $i \in [0, t-1]$, the reduction can interpolate in the exponent to compute the remaining points $\tilde{Y}_t, \tilde{Y}_{t+1}, \dots, \tilde{Y}_n$. The reduction similarly computes $Y_i = g^{f_y(i)}$ using the tuples provided by $\mathcal{C}$ (in the exponent) for all $i \in [0, n]$. It then sends I to the adversary $\mathcal{A}$. $\mathcal{B}$ also internally initializes $\mathcal{C}, Q_1, Q_2 := \emptyset$ and then does the following:

- For any O_{Corrupt} query with index $i \in [n]$ by $\mathcal{A}$, the reduction $\mathcal{B}$ responds by using the discrete logarithm oracle provided by $\mathcal{C}$, i.e., it submits $\tilde{Y}_i$ to O_{dlog} , receives t_i , and returns $(f_x(i), t_i)$ to $\mathcal{A}$. It also internally updates $\mathcal{C} \leftarrow \mathcal{C} \cup \{(i, t_i)\}$.
- For any O_1 query with message $\mu \in \mathbb{Z}_p$, the oracle first checks if there exists a tuple $(\mu, r) \in Q_1$. If it does, then it returns $A_\mu = g^r$. Otherwise, it picks a random $r \leftarrow \mathbb{Z}_p^*$, internally updates $Q_1 \leftarrow Q_1 \cup \{(\mu, r)\}$ and returns g^r .

- For any $\mathcal{O}_2$ query with message $(\mu, i) \in \mathbb{Z}_p$, the oracle aborts if $(i, \cdot) \in \mathcal{C}$. Otherwise, it checks if a tuple $(\mu, r) \in Q_1$ exists. If it does not exist, it picks a random $r \leftarrow \mathbb{Z}_p$, internally updates $Q_1 \leftarrow Q_1 \cup \{(\mu, r)\}$. It then outputs $(A_\mu, B_{\mu,i}) := (g^r, (X_i \cdot Y_i^\mu)^r)$. It also internally updates $Q_2 \leftarrow Q_2 \cup \{(\mu, i)\}$.

Finally, $\mathcal{A}$ outputs a tuple (μ^*, h^*, H^*) , where it made q_1 queries to $\mathcal{O}_1$, $q_{2,i}$ queries to $\mathcal{O}_2$ with $(\cdot, i)$ and $q_{\mathcal{C}}$ queries to oracle $\mathcal{O}_{\text{Corrupt}}$. However, because $\mathcal{A}$ is algebraic it provides values $v_R, w_R, \{v_\mu, w_\mu\}_{(\mu, \cdot) \in Q_1}, \{v_{\mu,i}, w_{\mu,i}\}_{(\mu, i) \in Q_2}$ such that:

$$\begin{aligned} h^* &= g^{v_R} \cdot \prod_{(\mu, \cdot) \in Q_1} (A_\mu)^{v_\mu} \cdot \prod_{(\mu, i) \in Q_2} (B_{\mu,i})^{v_{\mu,i}} \\ H^* &= g^{w_R} \cdot \prod_{(\mu, \cdot) \in Q_1} (A_\mu)^{w_\mu} \cdot \prod_{(\mu, i) \in Q_2} (B_{\mu,i})^{w_{\mu,i}}. \end{aligned}$$

We can rearrange the above equations and represent them in terms of the polynomial f_y and values $a_R, b_R, a_x, b_x, a_1, \dots, a_n, b_1, \dots, b_n$ (known to $\mathcal{B}$) as:

$$h^* = g^{a_R} \cdot X^{a_x} \cdot \prod_{i \in [0, n]} g^{f_y(i) \cdot a_i} \quad (5)$$

$$H^* = g^{b_R} \cdot X^{b_x} \cdot \prod_{i \in [0, n]} g^{f_y(i) \cdot b_i}. \quad (6)$$

Note that because polynomial $f_x(X)$ is chosen by $\mathcal{B}$, we can represent all X_i in terms of X to some power a_x . This already looks like the starting point of the equation in the proofs in the algebraic group model for the standard PS assumption. The main difference is that instead of having a single unknown value y , the equations are defined with respect to a polynomial $f_y(X)$. Without loss of generality, we assume that $\mathcal{A}$ made all $(t-1)$ allowable queries to $\mathcal{O}_{\text{Corrupt}}$ (otherwise, $\mathcal{A}$ can make those queries instead). Thus, the reduction knows $(t-1)$ distinct points on $f_y(X)$. Let us use $(t_1, f_y(t_1)), \dots, (t_{t-1}, f_y(t_{t-1}))$ to denote them. Note that $\mathcal{B}$ is missing one point to interpolate f_y and solve all the discrete logarithms $(T_0, \tilde{T}_0), \dots, (T_{t-1}, \tilde{T}_{t-1})$. However, since the known points are distinct, they interpolate a unique polynomial $q_y(X)$ of degree $(t-2)$ for which we can represent the polynomial $f_y(X)$ as

$$f_y(X) := q_y(X) + \xi \cdot u_y(X),$$

where $u_y(X) := \prod_{i=1}^{t-1} (X - t_i)$ and ξ is a scalar that is the remaining degree of freedom and unknown to $\mathcal{B}$. Note that it knows $q_y(X)$ and $u_y(X)$. Hence, we can substitute into equations (5) and (6), and re-write as

$$\begin{aligned} h^* &= g^{a'_R + a_x \cdot f_x(0) + \xi \cdot a'_y} \\ H^* &= g^{b'_R + b_x \cdot f_x(0) + \xi \cdot b'_y}, \end{aligned}$$

where

$$\begin{aligned} a'_R &= a_R + \sum_{i \in [0, n]} q_y(i) \cdot a_i & a'_y &= \sum_{i \in [0, n]} u_y(i) \cdot a_i \\ b'_R &= b_R + \sum_{i \in [0, n]} q_y(i) \cdot b_i & b'_y &= \sum_{i \in [0, n]} u_y(i) \cdot b_i. \end{aligned}$$

The reduction $\mathcal{B}$ can now use the same steps as in the standard PS signatures to obtain ξ . With this, the reductions interpolates polynomial $f_y(X)$ using $f_y(X) := q_y(X) + \xi \cdot u_y(X)$ and returns $f_y(0), f_y(1), \dots, f_y(t)$ to C . Note that since $q_C \leq t$, it follows that $\mathcal{B}$ only queries the discrete logarithm oracle at most t times, and hence the output is a valid solution to the one-more co-discrete logarithm problem. The same works analogously if the target is $\tilde{X}$. $\square$

Theorem 5.3 (One-more unforgeability). *Assuming the (t, n) -threshold PS problem is hard and Π_R and Π_{PS} online knowledge extractable, Construction 2 is one-more unforgeable.*

Proof. We prove this theorem via a reduction to the hardness of the (t, n) -threshold PS problem. Formally, suppose a PPT adversary $\mathcal{A}$ has non negligible advantage against one-more unforgeability of the NITBS scheme; we will then build a reduction $\mathcal{B}$ that will break the (t, n) -threshold PS assumption with non-negligible advantage as follows:

- Given $(p, g, \tilde{g}, e, \tilde{X} := \tilde{g}^{f_x(0)}, \tilde{Y} := \tilde{g}^{f_y(0)}, \{\tilde{X}_i := \tilde{g}^{f_x(i)}, \tilde{Y}_i := \tilde{g}^{f_y(i)}\}_{i \in [n]})$ from the challenger C , the reduction $\mathcal{B}$ samples $\beta_1, \beta_2, z \leftarrow \mathbb{Z}_p^*$ and sends the public parameters $pp := (p, g, \tilde{g}, e, g_1 := g^{\beta_1}, g_2 := g^{\beta_2}, \text{crs}_R, \text{crs}_{PS})$ and $\text{mpk} := (\tilde{X}, \tilde{Y}_1 := \tilde{Y}, \tilde{Y}_2 := \tilde{Y}^z)$ to the adversary $\mathcal{A}$. Here crs_R and crs_{PS} are the honestly generated reference strings for languages $\mathcal{L}_R$ and $\mathcal{L}_{PS}$ respectively. $\mathcal{B}$ internally initializes $\mathcal{K}, \mathcal{C}, Q := \emptyset$.
- For any O_{Corrupt} query with index $i \in [n]$ by $\mathcal{A}$, the reduction $\mathcal{B}$ responds by using its own corruption oracle O_{Corrupt}^{PS} for the (t, n) -threshold PS problem. It also internally updates $\mathcal{C} \leftarrow \mathcal{C} \cup \{i\}$.
- For any O_{Issue} query of the form (i, pk_R, c) by $\mathcal{A}$, the reduction $\mathcal{B}$ responds as follows:
 - It returns $\perp$ if $i \in \mathcal{C}$ and continues otherwise.
 - It checks if there exists a tuple $(\text{pk}_R, c, \kappa_{\text{pk}_R, c}) \in Q$. If not, it internally queries oracle H_G on input (pk_R, c) , receiving group element h
 - It checks if a tuple $(\text{pk}_R, \alpha) \in \mathcal{K}$ exists. If not, it runs the extractor $\mathcal{E}_R$ on $(x := (g_1, \text{pk}'_R), \pi_R)$ to obtain the witness α satisfying $\text{pk}'_R = g_1^{1/\alpha}$. It also internally updates $\mathcal{K} \leftarrow \mathcal{K} \cup \{(\text{pk}_R, \alpha)\}$.
 - It computes $\mu := \kappa_{\text{pk}_R, c} + z\alpha$ and queries its own O_2^{PS} with (i, μ) , receiving group element $h^{f_x(i) + f_y(i) \cdot \mu}$, which it returns to $\mathcal{A}$.
- For any H_Z query of the form (pk_R, c) from $\mathcal{A}$, $\mathcal{B}$ picks a random element $\kappa_{\text{pk}_R, c} \leftarrow \mathbb{Z}_p$ and returns it. It also internally updates $Q \leftarrow Q \cup \{(\text{pk}_R, c, \kappa_{\text{pk}_R, c})\}$.
- For any H_G query of the form (pk_R, c) from $\mathcal{A}$, $\mathcal{B}$ responds as follows:
 - It checks if there exists a tuple $(\text{pk}_R, c, \kappa_{\text{pk}_R, c}) \in Q$. If not, it internally queries oracle H_Z on input (pk_R, c) .
 - It checks if a tuple $(\text{pk}_R, \alpha) \in \mathcal{K}$ exists. If not, it runs the extractor $\mathcal{E}_R$ on $(x := (g_1, \text{pk}'_R), \pi_R)$ to obtain the witness α satisfying $\text{pk}'_R = g_1^{1/\alpha}$. It also internally updates $\mathcal{K} \leftarrow \mathcal{K} \cup \{(\text{pk}_R, \alpha)\}$.
 - It computes $\mu := \kappa_{\text{pk}_R, c} + z\alpha$ and queries its own O_1^{PS} with μ , receiving group element h , which it returns to $\mathcal{A}$.
- Finally, after making $\ell = \text{poly}(\lambda)$ queries, when $\mathcal{A}$ outputs its (one-more) forgery $\{m_i, \sigma_i\}_{i \in [\ell+1]}$, $\mathcal{B}$ does the following:

- For every $i \in [\ell + 1]$, it runs the extractor $\mathcal{E}_{\text{PS}}$ on $(x := (\tilde{X}, \tilde{Y}_1, \tilde{Y}_2, m_i), \pi_{\text{PS}}^{(i)})$ to obtain the witness $\varphi_{1,i}, \varphi_{2,i}, \kappa_i, \alpha_i$ satisfying $\mathcal{L}_{\text{PS}}$.
- If not all $(\kappa_i + z\alpha_i)$ are distinct, then $\mathcal{B}$ aborts. Otherwise, since $L = \{(\text{pk}_R, c) : |Q_{\text{pk}_R, c} \cup \mathcal{C}| \geq t + 1\}$ and $\ell \leq L$, by a simple pigeonhole argument there must exist at least one $i^* \in [L]$ such that the corresponding message $\mu^* := (\kappa_{i^*} + z\alpha_{i^*})$ was queried by $\mathcal{B}$ to $\mathcal{O}_2$ less than $(t - C)$ times, meaning that $(\mu^*, \varphi_1, \varphi_2)$ is a valid solution to the (t, n) -threshold PS problem and $\mathcal{B}$ can return it to $\mathcal{C}$.

Since, (φ_1, φ_2) is a valid PS signature on μ^* , $\mathcal{B}$ wins with probability

$$\text{Adv}_{\mathcal{B}}^{(t,n)\text{-PS}}(\lambda) \leq \text{Adv}_{\mathcal{A}}^{\text{OM-Unf}}(\lambda) + \text{Adv}_{\mathcal{A}}^{\text{KSnd,R}}(\lambda) + \text{Adv}_{\mathcal{A}}^{\text{KSnd,PS}}(\lambda) + \Pr[\text{abort}],$$

By assumption, $\text{Adv}_{\mathcal{A}}^{\text{KSnd,R}}(\lambda)$ and $\text{Adv}_{\mathcal{A}}^{\text{KSnd,PS}}(\lambda)$ are negligible, so to complete the proof it only remains to be shown that the probability that $\mathcal{B}$ aborts is also negligible. In particular, we analyze the condition that *not all* $(\kappa_i + z\alpha_i)$ are distinct. In other words, there exists some $i, j \in [\ell + 1]$ such that $\kappa_i + z\alpha_i = \kappa_j + z\alpha_j$. We can use the same arguments here as in the case of the proof of the non-threshold version (Theorem 5.3). Now, since a valid one-more forgery requires that all messages defined as $m_i := g_2^{1/(\kappa_i + \alpha_i)}$ be distinct, then we also have that $\kappa_i + \alpha_i \neq \kappa_j + \alpha_j$. Notice that since $z \in \mathbb{Z}_p^*$, $\kappa_i + z\alpha_i = \kappa_j + z\alpha_j$ implies that $\kappa_i \neq \kappa_j \Leftrightarrow \alpha_i \neq \alpha_j$ and since $\kappa_i + \alpha_i \neq \kappa_j + \alpha_j$, we get that $\kappa_i \neq \kappa_j$ and $\alpha_i \neq \alpha_j$. However, given such a pair (κ_i, α_i) and (κ_j, α_j) , $\mathcal{A}$ could easily compute $z := \frac{\kappa_j - \kappa_i}{\alpha_i - \alpha_j} \pmod{p}$ which would break the discrete log base $\tilde{Y}$. Therefore, $\Pr[\text{abort}] \leq \text{Adv}_{\mathcal{A}}^{\text{DLOG}}(\lambda)$.

□

Theorem 5.4 (Recipient blindness). *Assuming the qDDH problem is hard in $\mathbb{G}$ and Π_{PS} zero-knowledge, Construction 2 is recipient blind.*

Sketch. It is worth noting that since the final signature is similar to the non-threshold version in Construction 1, it follows that the proof of Theorem 4.4 for recipient blindness can also be reused. The idea is to simulate all zero-knowledge and then replace the messages in the challenged signatures with random group elements. Finally, all the challenged values given to the adversary are either simulated without the recipient's secret key or are random group elements.

□

Theorem 5.5 (Context blindness). *Assuming the qDDH problem is hard in $\mathbb{G}$ and Π_{PS} zero-knowledge, Construction 2 is context blind.*

Sketch. Follows by using the same steps as in the proof for the non-threshold version (Construction 1).

□

5.2 Efficiency

We provide some micro-benchmarks for the Issue, Obtain and Verify algorithms for our protocol in Table 2 to demonstrate its feasibility. All estimates are over the b1strs implementation of BLS12-381 curve, and were obtained using `zka.1c` [ECK⁺24] for an AWS EC2 m5.large instance (comparable to a mid-range home computer). On the same curve, the receiver's public key is around 112 bytes, the presignature size is 96 bytes and the final signature is around 192 bytes for a 256-bit scalar field.

# of operations		Threshold				
		$t = 1$	$t = 2$	$t = 3$	$t = 4$	$t = 5$
Issue	$6\times$			—— 0.59 ms ——		
Obtain	$4e + (5 + 2t)\times$	3.837 ms	4.036 ms	4.235 ms	4.435 ms	4.634 ms
Verify	$4e + 5\times$			—— 3.638 ms ——		

Table 2: The number of operations and the estimated time taken by each step of our non-interactive threshold blind signature scheme for threshold t . $\times$ refers to scalar multiplication in base group $\mathbb{G}$, $\tilde{\times}$ to the same operation in second base group $\tilde{\mathbb{G}}$, and e refers to pairing operations. A dash through a row indicates that times are the same across all values of t .

6 Extensions

In this section we discuss various extensions to our constructions.

6.1 Tagged NI(T)BS

We first notice that both of the above constructions can easily be extended to include a public message that does not change. This is the notion of tagged non-interactive blind signatures introduced in [Han23]. Since this is a relatively straightforward extension for both the construction and security model (in the threshold case), we omit the details and only sketch out the main idea here.

Recall that in both cases, the final blind signature is a non-interactive zero-knowledge proof of knowledge of a PS signature under message (κ, α) with $\text{pk}_R = g_1^{1/\alpha}$ and $\kappa = H_{\mathbb{Z}}(\text{pk}_R, c)$. The idea for the tagged versions of constructions 1 and 2 is to sign one more element in the PS signature, i.e., (κ, α, τ) instead of (κ, α) where τ is the public message. The only differences are that the signer's public key consists of an additional element $\tilde{Y}_3 = \tilde{g}^{y_3}$ and that the statement for language $\mathcal{L}_{\text{PS}}$ must be tweaked slightly so that

$$e(\psi_1, \tilde{X}) \cdot e(\psi_1, \tilde{Y}_1)^\kappa \cdot e(\psi_1, \tilde{Y}_2)^\alpha \cdot e(\psi_1, \tilde{Y}_3)^\tau = e(\psi_2, \tilde{g}).$$

Moreover, since τ is given as part of the (public) statement, the size of the final blind signature σ does not change. The communication size also does not change since the signer can include τ in A by computing $A = h^{x+y_1\cdot\kappa+y_3\cdot\tau} \cdot g_1^r$. Only the signer's public key increases by a single $\tilde{\mathbb{G}}$ element.

6.2 NITBS with verifiable presignatures

We observe that a simple extension to non-interactive threshold blind signatures can allow the recipient to verify the validity of presignature shares. In particular, our NITBS formalization can be extended so that the recipient is also able to verify the correctness of *presignature shares*. This is often necessary in applications where signers have incentives to misbehave (e.g., when recipients pay for presignatures). Given that, in the threshold setting, recipients will have to aggregate multiple presignature shares, being able to validate them first offers a significant advantage by avoiding trial-and-error detection of spurious shares.

We begin by introducing a new algorithm to the syntax of NITBS:

- $\text{VerifyPartial}(\text{sk}_R, \text{mpk}, i, \bar{\sigma}_i, c) \rightarrow 1/0$. Given the recipient's secret key $\text{sk}_R \in \mathcal{SK}_R$, the master public key $\text{mpk} \in \mathcal{PK}$, an index $i \in [n]$, presignature share $\bar{\sigma}_i$ and context c , the deterministic verification algorithm outputs 1 (accept) or 0 (reject).

With VerifyPartial , the recipient can verify the validity of a presignature share $\bar{\sigma}_i$. We will now show how to extend Construction 2 to provide this property. In the first step, we modify how the master public key mpk is generated. Given the shares $([x]_i, [y_1]_i, [y_2]_i) \leftarrow \text{Share}_{t,n}((x, y_1, y_2))$ of the secrets x, y_1, y_2 , the master public key includes $\tilde{X}, \tilde{Y}_1, \tilde{Y}_2$ (same as in the original construction), as well as partial keys $\tilde{X}_i := \tilde{g}^{[x]_i}, \tilde{Y}_{1,i} := \tilde{g}^{[y_1]_i}, \tilde{Y}_{2,i} := \tilde{g}^{[y_2]_i}$, i.e., $\text{mpk} = (\tilde{X}, \tilde{Y}_1, \tilde{Y}_2, \{\tilde{X}_i, \tilde{Y}_{1,i}, \tilde{Y}_{2,i}\}_{i \in [n]})$. Moreover, we will use the notation mpk_i to denote the tuple $(\tilde{X}_i, \tilde{Y}_{1,i}, \tilde{Y}_{2,i})$.

Now, a presignature in Construction 2 is of the form $\bar{\sigma}_i = (A_i, B_i)$, where in case of an honest signer we have $A = h^{[x]_i + [y_1]_i \cdot \kappa} \cdot g_1^r, B = (\text{pk}_R)^{-r} h^{[y_2]_i}, h = H_{\mathbb{G}}(\text{pk}_R, c)$ and $\kappa = H_{\mathbb{Z}}(\text{pk}_R, c)$. It is easy to see that from this, the recipient can first reconstruct a PS signature (φ_1, φ_2) that is valid under public key mpk_i . Below, we provide the detailed steps.

$\text{VerifyPartial}(i, \text{mpk}, \bar{\sigma}_i = (A, B), \text{sk}_R = \alpha, c)$

- 1: $h := H_{\mathbb{G}}(g_1^\alpha, c)$
- 2: $\kappa := H_{\mathbb{Z}}(g_1^\alpha, c)$
- 3: $(\tilde{X}, \tilde{Y}_1, \tilde{Y}_2) := \text{mpk}_i$
- 4: $(\varphi_1, \varphi_2) := (h, (A \cdot B^\alpha))$
- 5: **return** $e(\varphi_2, \tilde{g}) \stackrel{?}{=} e(\varphi_1, \tilde{X} \cdot \tilde{Y}_1^\kappa \cdot \tilde{Y}_2^\alpha)$

Theorem 6.1 (One-more unforgeability). *Assuming the (t, n) -threshold PS problem is hard and Π_R and Π_{PS} online knowledge extractable, the above NITBS with verifiable presignature construction is one-more unforgeable (in the random oracle model).*

Sketch. The proof follows the same steps as the proof of Theorem 5.3 for the non-interactive threshold blind signatures (Construction 2). The idea is that the (t, n) -threshold PS problem already gives all the required elements for the mpk , and the reduction can proceed similarly. $\square$

Theorem 6.2 (Recipient blindness). *Assuming the sDDH and qDDH problems are hard in $\mathbb{G}$ and proof systems Π_{PS} zero-knowledge, the above NITBS with verifiable presignature construction is recipient blind (in the random oracle model and honest key models).*

Theorem 6.3 (Context blindness). *Assuming the sDDH and qDDH problems are hard in $\mathbb{G}$ and proof systems Π_{PS} zero-knowledge, the above NITBS with verifiable presignature construction is context blind (in the random oracle model and honest key models).*

It is worth noting that since there is no change in the presignatures and the signatures introduced by the presignature verification algorithm, blindness holds similarly to the blindness of Construction 2.

6.3 NI(T)BS with batch issuance

In a recent work, [BGY25] extended NIBS to the batched setting. Namely, given a single presignature, the user can derive a batch of message-signature pairs for some a priori fixed batch size N . Here we give a high-level overview of how our NI(T)BS scheme can also be batched. First, we notice that the message

$m := g_2^{1/(\kappa+\alpha)}$ can be seen as a PRF for key α and input κ (see [DY05]). Having this in mind, in the batched version, we allow the recipient to use messages $m_0 := g_2^{1/(\kappa+\alpha)}, \dots, m_{N-1} := g_1^{1/(\kappa+\alpha+(N-1))}$. This requires us to update the language $\mathcal{L}_{PS}$ as follows:

Language $\mathcal{L}'_{PS}$

Instance: Each instance x is interpreted as the issuer's public key $pk = (\tilde{X}, \tilde{Y}_1, \tilde{Y}_2) \in \tilde{\mathbb{G}}^3$, a group element $m \in \tilde{\mathbb{G}}$ and an integer $N > 0$.

Witness: Witness w consists of group elements $\Phi = (\varphi_1, \varphi_2) \in \mathbb{G}^2$, scalars $(\kappa, \alpha) \in (\mathbb{Z}_p^*)^2$ and integer $i \in \mathbb{N}$.

Membership: w is a valid witness for x if the following are satisfied:

- ▷ Φ is a valid (secret) PS signature for the message (κ, α)
- $e(\varphi_1, \tilde{X}) \cdot e(\varphi_1, \tilde{Y}_1)^\kappa \cdot e(\varphi_1, \tilde{Y}_2)^\alpha = e(\varphi_2, \tilde{g})$
- ▷ and m is a valid NIBS message constructed from (κ, α)
- $\wedge m^{\kappa+\alpha+i} = g_2 \wedge i \in [0, N-1]$

In order to compose cleanly with a range proof system for proving $i \in [0, N-1]$, we use a different instantiation for the NIZK proof system π_{PS} than the one presented in Section 4.2. We provide the details of our instantiation further below. It is worth noting that with the modified approach, the presignature and keys remain the same in the batched version, and only the signature size increases logarithmically in the batch size, N .

6.3.1 Concrete NIZK for NI(T)BS with batch issuance

The proof for language $\mathcal{L}'_{PS}$ comprises of the following:

- A proof of knowledge π_1 of a PS signature Φ under messages κ and α , which are given as Pedersen commitments $\tilde{C}_1 = \tilde{g}^{\tilde{r}_1} \tilde{Y}_1^\kappa$ and $\tilde{C}_2 = \tilde{g}^{\tilde{r}_2} \tilde{Y}_2^\alpha$ for $(\tilde{r}_1, \tilde{r}_2) \leftarrow \$ (\mathbb{Z}_p^*)^2$.
- A proof of discrete logarithm equivalence π_2 between commitments $\tilde{C}_1, \tilde{C}_2$ and commitments $C_1 = g^{r_1} g_2^\kappa$ and $C_2 = g^{r_2} g_2^\alpha$ for $(r_1, r_2) \leftarrow \$ (\mathbb{Z}_p^*)^2$.
- A proof of discrete logarithm equivalence π_3 between the message committed in $C_{123} := C_1 \cdot C_2 \cdot C_3 = g^{r_1+r_2+r_3} g_2^{\kappa+\alpha+i}$ and $m = g_2^{1/(\kappa+\alpha)}$. Here $C_3 = g^{r_3} g_2^i$.
- A range proof showing that C_3 is a commitment to an integer in the interval $[0, N-1]$.

We now describe each proof separately.

- (i) We start with proof π_1 , which is inspired by the original proof of knowledge from [PS16]. Given a PS signature $\Phi = (\varphi_1, \varphi_2)$ ¹¹, the prover first picks random $(v, \rho) \leftarrow \$ \mathbb{Z}_p^2$ and computes $\psi_1 := \varphi_1^\rho$, and $\psi_2 := (\varphi_2 \cdot \varphi_1^v)^\rho$. The proof is a proof of knowledge of $\varepsilon = v - \tilde{r}_1 - \tilde{r}_2$, such that

¹¹ Here the recipient need not re-randomize the derived PS signature under ρ as it did in Figures 4 and 5.

$$\begin{aligned}
e(\psi_1, \tilde{g})^\varepsilon &= e(\psi_2, \tilde{g}) \cdot e(\psi_1, \tilde{X})^{-1} \cdot e(\psi_1, \tilde{C}_1)^{-1} \cdot e(\psi_1, \tilde{C}_2)^{-1} \\
&= e(\psi_2, \tilde{g}) \cdot e(\psi_1, \tilde{X} \cdot \tilde{C}_1 \cdot \tilde{C}_2)^{-1}.
\end{aligned}$$

In particular, this can be viewed as a simple proof of knowledge of the discrete logarithm ε of $E_2 := e(\psi_2, \tilde{g}) \cdot e(\psi_1, \tilde{X} \cdot \tilde{C}_1 \cdot \tilde{C}_2)^{-1}$ to the base $E_1 := e(\psi_1, \tilde{g})$, i.e., $E_2 = E_1^\varepsilon$ in the target group $\hat{\mathbb{G}}$. Therefore, to compute this proof, the prover first computes commitment $T := E_1^{\varepsilon'}$ and the challenge $c_1 := H(E_1, E_2, T)$. The proof π_1 consists of (c_1, s_1) , where $s_1 := \varepsilon' - c_1 \cdot \varepsilon$. To verify the proof, given (pp, pk, $\tilde{C}_1, \tilde{C}_2, \psi_1, \psi_2$), the verifier reconstructs E_1, E_2 and computes $T' := E_1^{s_1} \cdot E_2^{c_1}$. Then accepts π_1 if $H(E_1, E_2, T') = c_1$.

- (ii) The second proof π_2 is a proof of discrete logarithm equivalence for commitments $\tilde{C}_1, \tilde{C}_2$ and C_1, C_2 . In other words, it must hold that $\text{dlog}_{\tilde{Y}_1}(\tilde{C}_1 \cdot \tilde{g}^{-\tilde{r}_1}) = \text{dlog}_{g_2}(C_1 \cdot g^{-r_1}) (= \kappa)$ and similarly $\text{dlog}_{\tilde{Y}_2}(\tilde{C}_2 \cdot \tilde{g}^{-\tilde{r}_2}) = \text{dlog}_{g_2}(C_2 \cdot g^{-r_2}) (= \alpha)$. This is a combination of simple Schnorr type proofs, where we prove the knowledge of $r_1, r_2, \tilde{r}_1, \tilde{r}_2, \kappa, \alpha$. Specifically, the prover computes commitments $T_1 := g^{r_1'} g_2^{\kappa'}$, $\tilde{T}_1 := \tilde{g}^{\tilde{r}_1'} \tilde{Y}_1^{\kappa'}$, $T_2 := g^{r_2'} g_2^{\alpha'}$ and $\tilde{T}_2 := \tilde{g}^{\tilde{r}_2'} \tilde{Y}_2^{\alpha'}$ for $(r_1', r_2', \tilde{r}_1', \tilde{r}_2', \kappa', \alpha') \leftarrow \$ (\mathbb{Z}_p^*)^6$ and the challenge $c_2 := H(g, g_2, \tilde{g}, \tilde{Y}_1, \tilde{Y}_2, C_1, \tilde{C}_1, C_2, \tilde{C}_2, T_1, \tilde{T}_1, T_2, \tilde{T}_2)$. The proof π_2 consists of $(c_2, s_{2,\kappa}, s_{2,r_1}, s_{2,\tilde{r}_1}, s_{2,\alpha}, s_{2,r_2}, s_{2,\tilde{r}_2})$ where each $s_{2,\circ} := \circ' - c_2 \circ$ with $\circ$ replaced appropriately by $\kappa, r_1, \tilde{r}_1, \alpha, r_2$ or $\tilde{r}_2$. To verify the proof, given (pp, pk, $C_1, \tilde{C}_1, C_2, \tilde{C}_2$), the verifier computes $T_1' := g^{s_{2,r_1}} \cdot g_2^{s_{2,\kappa}} \cdot C_1^{c_2}$, $\tilde{T}_1' := \tilde{g}^{s_{2,\tilde{r}_1}} \cdot \tilde{Y}_1^{s_{2,\kappa}} \cdot \tilde{C}_1^{c_2}$, $T_2' := g^{s_{2,r_2}} \cdot g_2^{s_{2,\alpha}} \cdot C_2^{c_2}$ and $\tilde{T}_2' := \tilde{g}^{s_{2,\tilde{r}_2}} \cdot \tilde{Y}_2^{s_{2,\alpha}} \cdot \tilde{C}_2^{c_2}$. Then accepts π_2 if $H(g, \tilde{g}, \tilde{Y}_1, \tilde{Y}_2, C_1, \tilde{C}_1, C_2, \tilde{C}_2, T_1', \tilde{T}_1', T_2', \tilde{T}_2') = c_2$.
- (iii) The proof π_3 is similarly a proof of the discrete logarithm equivalence. However, instead of using each commitment separately, the statement now considers the commitment $C_{123} := C_1 \cdot C_2 \cdot C_3$, which is a Pedersen commitment to the message $\mu := (\kappa + \alpha + i)$. Thus, in this proof, the prover demonstrates knowledge of the randomness $t := r_1 + r_2 + r_3$ used in C_{123} and the message μ committed in C_{123} , which corresponds to the value $m = g_2^{1/\mu}$. Thus, it must hold that $\text{dlog}_{g_2}(m) \cdot \text{dlog}_{g_2}(C_{123} \cdot g^{-t}) = 1$. So, the prover computes commitments $T_{123} := g^{t'} g_2^{\mu'}$ and $T_\mu := m^{\mu'}$ for $(t', \mu') \leftarrow \$ (\mathbb{Z}_p^*)^2$ and the challenge $c_3 := H(g, g_2, C_1, C_2, C_3, m, T_{123}, T_\mu)$. The proof π_3 consists of $(c_3, s_{3,t}, s_{3,\mu})$ where $s_{3,t} := t' - c_3 t$ (similarly for $s_{3,\mu}$). To verify the proof, given (pp, C_1, C_2, C_3), the verifier computes $T_{123}' := g^{s_{3,t}} \cdot g_2^{s_{3,\mu}} \cdot (C_1 \cdot C_2 \cdot C_3)^{c_3}$ and $T_\mu' := g_2^{c_3} \cdot m^{s_{3,\mu}}$. Then accepts π_3 if $H(g, g_2, C_1, C_2, C_3, m, T_{123}', T_\mu') = c_3$.
- (iv) The range proof π_4 can be implemented generically by employing the Bulletproof system [BBB⁺18]. Alternatively, let the batch size $N = 2^k$ and $i_0 \dots i_{k-1}$ be the bit decomposition of index i . Then, the prover can compute commitments $C_{3,j} := g^{r_{3,j}} g_2^{i_j}$, such that $\prod_{j=0}^{k-1} (C_{3,j})^{2^j} = C_3$, i.e., given all $C_{3,j}$. It can then prove, using standard disjunctive proof techniques, that for each $j \in [0, k-1]$ the commitment $C_{3,j}$ is a commitment to a bit, which is enough to prove that C_3 is a commitment to a value $0 \leq i < N = 2^k$.

Summarizing all the above, the proof π_R consists of 2 elements (c_R, s_R) in $\mathbb{Z}_p^*$; and the proof π_{PS} consists of the randomized PS signature (ψ_1, ψ_2) , Pedersen commitments $C_1, C_2, C_3, \tilde{C}_1, \tilde{C}_2$ and proofs $\pi_1 := (c_1, s_1)$, $\pi_2 := (c_2, s_{2,\kappa}, s_{2,r_1}, s_{2,\tilde{r}_1}, s_{2,\alpha}, s_{2,r_2}, s_{2,\tilde{r}_2})$, $\pi_3 := (c_3, s_{3,t}, s_{3,\mu})$. Therefore, π_{PS} consists of 5 (+2 lg N + 4 for Bulletproofs) elements in $\mathbb{G}$, 2 elements in $\hat{\mathbb{G}}$ and 12 (+5 for Bulletproofs) elements in $\mathbb{Z}_p^*$. As previously noted, the presignature and keys remain the same in the batched version, and only the signature size increases logarithmically in the batch size.

Acknowledgements. The authors thank the anonymous reviewer who provided useful insight regarding the concrete NIZK protocol. Foteini Baldimtsi and Aayush Yadav were supported by NSF awards 2143287 and 2247304. Lucjan Hanzlik was supported by Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) award 545066907.

References

- [Abe01] Masayuki Abe. A secure three-move blind signature scheme for polynomially many signatures. In Birgit Pfitzmann, editor, *Advances in Cryptology – EUROCRYPT 2001*, volume 2045 of *Lecture Notes in Computer Science*, pages 136–151, Innsbruck, Austria, May 6–10, 2001. Springer Berlin Heidelberg, Germany.
- [ABM15] Michel Abdalla, Fabrice Benhamouda, and Philip MacKenzie. Security of the J-PAKE password-authenticated key exchange protocol. In *2015 IEEE Symposium on Security and Privacy*, pages 571–587, San Jose, CA, USA, May 17–21, 2015. IEEE Computer Society Press.
- [AKSY22] Shweta Agrawal, Elena Kirshanova, Damien Stehlé, and Anshu Yadav. Practical, round-optimal lattice-based blind signatures. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022: 29th Conference on Computer and Communications Security*, pages 39–53, Los Angeles, CA, USA, November 7–11, 2022. ACM Press.
- [App21] Apple. icloud private relay overview. https://www.apple.com/icloud/docs/iCloud_Private_Relay_Overview_Dec2021.pdf, 2021. Accessed: 2024-04-29.
- [ASM06] Man Ho Au, Willy Susilo, and Yi Mu. Constant-size dynamic k-TAA. In Roberto De Prisco and Moti Yung, editors, *SCN 06: 5th International Conference on Security in Communication Networks*, volume 4116 of *Lecture Notes in Computer Science*, pages 111–125, Maiori, Italy, September 6–8, 2006. Springer Berlin Heidelberg, Germany.
- [BBB⁺18] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE Symposium on Security and Privacy*, pages 315–334, San Francisco, CA, USA, May 21–23, 2018. IEEE Computer Society Press.
- [BBS04] Dan Boneh, Xavier Boyen, and Hovav Shacham. Short group signatures. In Matthew Franklin, editor, *Advances in Cryptology – CRYPTO 2004*, volume 3152 of *Lecture Notes in Computer Science*, pages 41–55, Santa Barbara, CA, USA, August 15–19, 2004. Springer Berlin Heidelberg, Germany.
- [BCC04] Ernest F. Brickell, Jan Camenisch, and Liqun Chen. Direct anonymous attestation. In Vijayalakshmi Atluri, Birgit Pfitzmann, and Patrick McDaniel, editors, *ACM CCS 2004: 11th Conference on Computer and Communications Security*, pages 132–145, Washington, DC, USA, October 25–29, 2004. ACM Press.
- [BCGY24] Foteini Baldimtsi, Jiaqi Cheng, Rishab Goyal, and Aayush Yadav. Non-interactive blind signatures: Post-quantum and stronger security. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024, Part II*, volume 15485 of *Lecture Notes in Computer Science*, pages 70–104, Kolkata, India, December 9–13, 2024. Springer, Singapore, Singapore.

- [BFL20] Balthazar Bauer, Georg Fuchsbauer, and Julian Loss. A classification of computational assumptions in the algebraic group model. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020, Part II*, volume 12171 of *Lecture Notes in Computer Science*, pages 121–151, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland.
- [BGY25] Foteini Baldimtsi, Rishab Goyal, and Aayush Yadav. Batched & Non-interactive Blind Signatures from Lattices. Cryptology ePrint Archive, Paper 2025/1771, 2025.
- [BHNY25] Foteini Baldimtsi, Lucjan Hanzlik, Quan Nguyen, and Aayush Yadav. Non-interactive anonymous tokens with private metadata bit. Cryptology ePrint Archive, Report 2025/430, 2025.
- [BL13] Foteini Baldimtsi and Anna Lysyanskaya. Anonymous credentials light. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *ACM CCS 2013: 20th Conference on Computer and Communications Security*, pages 1087–1098, Berlin, Germany, November 4–8, 2013. ACM Press.
- [BLNS23] Ward Beullens, Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Gregor Seiler. Lattice-based blind signatures: Short, efficient, and round-optimal. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023: 30th Conference on Computer and Communications Security*, pages 16–29, Copenhagen, Denmark, November 26–30, 2023. ACM Press.
- [Bol03] Alexandra Boldyreva. Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In Yvo Desmedt, editor, *PKC 2003: 6th International Workshop on Theory and Practice in Public Key Cryptography*, volume 2567 of *Lecture Notes in Computer Science*, pages 31–46, Miami, FL, USA, January 6–8, 2003. Springer Berlin Heidelberg, Germany.
- [CAHL⁺22] Rutchathon Chairattana-Apirom, Lucjan Hanzlik, Julian Loss, Anna Lysyanskaya, and Benedikt Wagner. PI-cut-choo and friends: Compact blind signatures via parallel instance cut-and-choose and more. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part III*, volume 13509 of *Lecture Notes in Computer Science*, pages 3–31, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.
- [CATZ24] Rutchathon Chairattana-Apirom, Stefano Tessaro, and Chenzhi Zhu. Pairing-free blind signatures from CDH assumptions. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part I*, volume 14920 of *Lecture Notes in Computer Science*, pages 174–209, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland.
- [CDL16] Jan Camenisch, Manu Drijvers, and Anja Lehmann. Anonymous attestation using the strong Diffie Hellman assumption revisited. Cryptology ePrint Archive, Report 2016/663, 2016.
- [CDVW24] S. Celi, A. Davidson, S. Valdez, and C. A. Wood. Privacy pass issuance protocols. RFC 9578, IETF, June 2024. Proposed Standard RFC.
- [CGT06] Sébastien Canard, Matthieu Gaud, and Jacques Traoré. Defeating malicious servers in a blind signatures based voting system. In Giovanni Di Crescenzo and Avi Rubin, editors,

FC 2006: 10th International Conference on Financial Cryptography and Data Security, volume 4107 of *Lecture Notes in Computer Science*, pages 148–153, Anguilla, British West Indies, February 27 – March 2, 2006. Springer Berlin Heidelberg, Germany.

- [Cha83] David Chaum. Blind signature system. In David Chaum, editor, *Advances in Cryptology – CRYPTO’83*, page 153, Santa Barbara, CA, USA, 1983. Plenum Press, New York, USA.
- [CKM⁺23] Elizabeth C. Crites, Chelsea Komlo, Mary Maller, Stefano Tessaro, and Chenzhi Zhu. Snow-blind: A threshold blind signature in pairing-free groups. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part I*, volume 14081 of *Lecture Notes in Computer Science*, pages 710–742, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.
- [DGS⁺18] Alex Davidson, Ian Goldberg, Nick Sullivan, George Tankersley, and Filippo Valsorda. Privacy pass: Bypassing internet challenges anonymously. *Proceedings on Privacy Enhancing Technologies*, 2018(3):164–180, July 2018.
- [DIW24] A. Davidson, J. Iyengar, and C. A. Wood. The privacy pass architecture. RFC 9576, IETF, June 2024. Informational RFC.
- [dPK22] Rafaël del Pino and Shuichi Katsumata. A new framework for more efficient round-optimal lattice-based (partially) blind signature via trapdoor sampling. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part II*, volume 13508 of *Lecture Notes in Computer Science*, pages 306–336, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.
- [DY05] Yevgeniy Dodis and Aleksandr Yampolskiy. A verifiable random function with short proofs and keys. In Serge Vaudenay, editor, *PKC 2005: 8th International Workshop on Theory and Practice in Public Key Cryptography*, volume 3386 of *Lecture Notes in Computer Science*, pages 416–431, Les Diablerets, Switzerland, January 23–26, 2005. Springer Berlin Heidelberg, Germany.
- [ECK⁺24] Jens Ernstberger, Stefanos Chaliasos, George Kadianakis, Sebastian Steinhorst, Philipp Jovanovic, Arthur Gervais, Benjamin Livshits, and Michele Orrù. zk-bench: A toolset for comparative evaluation and performance benchmarking of SNARKs. In Clemente Galdi and Duong Hieu Phan, editors, *SCN 24: 14th International Conference on Security in Communication Networks, Part I*, volume 14973 of *Lecture Notes in Computer Science*, pages 46–72, Amalfi, Italy, September 11–13, 2024. Springer, Cham, Switzerland.
- [FHS15] Georg Fuchsbaauer, Christian Hanser, and Daniel Slamanig. Practical round-optimal blind signatures in the standard model. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *Advances in Cryptology – CRYPTO 2015, Part II*, volume 9216 of *Lecture Notes in Computer Science*, pages 233–253, Santa Barbara, CA, USA, August 16–20, 2015. Springer Berlin Heidelberg, Germany.
- [FHS19] Georg Fuchsbaauer, Christian Hanser, and Daniel Slamanig. Structure-preserving signatures on equivalence classes and constant-size anonymous credentials. *Journal of Cryptology*, 32(2):498–546, April 2019.

- [Fis06] Marc Fischlin. Round-optimal composable blind signatures in the common reference string model. In Cynthia Dwork, editor, *Advances in Cryptology – CRYPTO 2006*, volume 4117 of *Lecture Notes in Computer Science*, pages 60–77, Santa Barbara, CA, USA, August 20–24, 2006. Springer Berlin Heidelberg, Germany.
- [FKL18] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018, Part II*, volume 10992 of *Lecture Notes in Computer Science*, pages 33–62, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.
- [GG14] Sanjam Garg and Divya Gupta. Efficient round optimal blind signatures. In Phong Q. Nguyen and Elisabeth Oswald, editors, *Advances in Cryptology – EUROCRYPT 2014*, volume 8441 of *Lecture Notes in Computer Science*, pages 477–495, Copenhagen, Denmark, May 11–15, 2014. Springer Berlin Heidelberg, Germany.
- [Gha17] Essam Ghadafi. Efficient round-optimal blind signatures in the standard model. In Aggelos Kiayias, editor, *FC 2017: 21st International Conference on Financial Cryptography and Data Security*, volume 10322 of *Lecture Notes in Computer Science*, pages 455–473, Sliema, Malta, April 3–7, 2017. Springer, Cham, Switzerland.
- [Goo24] Google. Private state tokens. <https://developers.google.com/privacy-sandbox/protectations/private-state-tokens>, 2024. Accessed: 2024-04-29.
- [GRS⁺11] Sanjam Garg, Vanishree Rao, Amit Sahai, Dominique Schröder, and Dominique Unruh. Round optimal blind signatures. In Phillip Rogaway, editor, *Advances in Cryptology – CRYPTO 2011*, volume 6841 of *Lecture Notes in Computer Science*, pages 630–648, Santa Barbara, CA, USA, August 14–18, 2011. Springer Berlin Heidelberg, Germany.
- [HAB⁺17] Ethan Heilman, Leen Alshenibr, Foteini Baldimtsi, Alessandra Scafuro, and Sharon Goldberg. TumbleBit: An untrusted bitcoin-compatible anonymous payment hub. In *ISOC Network and Distributed System Security Symposium – NDSS 2017*, San Diego, CA, USA, February 26 – March 1, 2017. The Internet Society.
- [Han23] Lucjan Hanzlik. Non-interactive blind signatures for random messages. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part V*, volume 14008 of *Lecture Notes in Computer Science*, pages 722–752, Lyon, France, April 23–27, 2023. Springer, Cham, Switzerland.
- [HBG16] Ethan Heilman, Foteini Baldimtsi, and Sharon Goldberg. Blindly signed contracts: Anonymous on-blockchain and off-blockchain bitcoin transactions. In Jeremy Clark, Sarah Meiklejohn, Peter Y. A. Ryan, Dan S. Wallach, Michael Brenner, and Kurt Rohloff, editors, *FC 2016 Workshops*, volume 9604 of *Lecture Notes in Computer Science*, pages 43–60, Christ Church, Barbados, February 26, 2016. Springer Berlin Heidelberg, Germany.
- [HKL19] Eduard Hauck, Eike Kiltz, and Julian Loss. A modular treatment of blind signatures from identification schemes. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019, Part III*, volume 11478 of *Lecture Notes in Computer Science*, pages 345–375, Darmstadt, Germany, May 19–23, 2019. Springer, Cham, Switzerland.

- [HPZ25] Lucjan Hanzlik, Eugenio Paracucchi, and Riccardo Zanotto. Non-interactive blind signatures from RSA assumption and more. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025, Part II*, volume 15602 of *Lecture Notes in Computer Science*, pages 365–394, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland.
- [HS14] Christian Hanser and Daniel Slamanig. Structure-preserving signatures on equivalence classes and their application to anonymous credentials. In Palash Sarkar and Tetsu Iwata, editors, *Advances in Cryptology – ASIACRYPT 2014, Part I*, volume 8873 of *Lecture Notes in Computer Science*, pages 491–511, Kaoshiung, Taiwan, R.O.C., December 7–11, 2014. Springer Berlin Heidelberg, Germany.
- [JLO97] Ari Juels, Michael Luby, and Rafail Ostrovsky. Security of blind digital signatures (extended abstract). In Burton S. Kaliski, Jr., editor, *Advances in Cryptology – CRYPTO’97*, volume 1294 of *Lecture Notes in Computer Science*, pages 150–164, Santa Barbara, CA, USA, August 17–21, 1997. Springer Berlin Heidelberg, Germany.
- [JS25] Corentin Jeudy and Olivier Sanders. Improved lattice blind signatures from recycled entropy. In Yael Tauman Kalai and Seny E. Kamara, editors, *Advances in Cryptology – CRYPTO 2025*, pages 477–513, Cham, 2025. Springer Nature Switzerland.
- [KLR21] Jonathan Katz, Julian Loss, and Michael Rosenberg. Boosting the security of blind signature schemes. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021, Part IV*, volume 13093 of *Lecture Notes in Computer Science*, pages 468–492, Singapore, December 6–10, 2021. Springer, Cham, Switzerland.
- [KLX22] Julia Kastner, Julian Loss, and Jiayu Xu. On pairing-free blind signature schemes in the algebraic group model. In Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe, editors, *PKC 2022: 25th International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 13178 of *Lecture Notes in Computer Science*, pages 468–497, Virtual Event, March 8–11, 2022. Springer, Cham, Switzerland.
- [KM15] Veronika Kuchta and Mark Manulis. Rerandomizable threshold blind signatures. In Moti Yung, Liehuang Zhu, and Yanjiang Yang, editors, *Trusted Systems*, pages 70–89, Cham, 2015. Springer International Publishing.
- [KRB⁺24] Ioanna Karantaidou, Omar Renawi, Foteini Baldimtsi, Nikolaos Kamarinakis, Jonathan Katz, and Julian Loss. Blind multisignatures for anonymous tokens with decentralized issuance. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024: 31st Conference on Computer and Communications Security*, pages 1508–1522, Salt Lake City, UT, USA, October 14–18, 2024. ACM Press.
- [KSAP23] Hyoseung Kim, Olivier Sanders, Michel Abdalla, and Jong Hwan Park. Practical dynamic group signatures without knowledge extractors. *Designs, Codes and Cryptography*, 91(3):853–893, 2023.
- [LNÖ25] Anja Lehmann, Phillip Nazarian, and Cavit Özbay. Stronger security for threshold blind signatures. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025, Part II*, volume 15602 of *Lecture Notes in Computer Science*, pages 335–364, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland.

- [LNP22] Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Maxime Plançon. Efficient lattice-based blind signatures via gaussian one-time signatures. In Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe, editors, *PKC 2022: 25th International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 13178 of *Lecture Notes in Computer Science*, pages 498–527, Virtual Event, March 8–11, 2022. Springer, Cham, Switzerland.
- [MOR01] Silvio Micali, Kazuo Ohta, and Leonid Reyzin. Accountable-subgroup multisignatures: Extended abstract. In Michael K. Reiter and Pierangela Samarati, editors, *ACM CCS 2001: 8th Conference on Computer and Communications Security*, pages 245–254, Philadelphia, PA, USA, November 5–8, 2001. ACM Press.
- [MRFH24] Thibault Meunier, Cefan Daniel Rubin, and Armando Faz-Hernández. Privacy pass: upgrading to the latest protocol version. <https://blog.cloudflare.com/privacy-pass-standard>, 2024. Accessed: 2024-04-29.
- [Pas11] Rafael Pass. Limits of provable security from standard assumptions. In Lance Fortnow and Salil P. Vadhan, editors, *43rd Annual ACM Symposium on Theory of Computing*, pages 109–118, San Jose, CA, USA, June 6–8, 2011. ACM Press.
- [PS96] David Pointcheval and Jacques Stern. Provably secure blind signature schemes. In Kwangjo Kim and Tsutomu Matsumoto, editors, *Advances in Cryptology – ASIACRYPT’96*, volume 1163 of *Lecture Notes in Computer Science*, pages 252–265, Kyongju, Korea, November 3–7, 1996. Springer Berlin Heidelberg, Germany.
- [PS00a] Birgit Pfitzmann and Ahmad-Reza Sadeghi. Anonymous fingerprinting with direct non-repudiation. In Tatsuaki Okamoto, editor, *Advances in Cryptology – ASIACRYPT 2000*, volume 1976 of *Lecture Notes in Computer Science*, pages 401–414, Kyoto, Japan, December 3–7, 2000. Springer Berlin Heidelberg, Germany.
- [PS00b] David Pointcheval and Jacques Stern. Security arguments for digital signatures and blind signatures. *Journal of Cryptology*, 13(3):361–396, June 2000.
- [PS16] David Pointcheval and Olivier Sanders. Short randomizable signatures. In Kazue Sako, editor, *Topics in Cryptology – CT-RSA 2016*, volume 9610 of *Lecture Notes in Computer Science*, pages 111–126, San Francisco, CA, USA, February 29 – March 4, 2016. Springer, Cham, Switzerland.
- [PVW24] T. Pauly, S. Valdez, and C. A. Wood. The privacy pass http authentication scheme. RFC 9577, IETF, June 2024. Standards Track RFC.
- [PZ13] Christian Paquin and Greg Zaverucha. U-prove cryptographic specification v1.1 (revision 3). Cryptographic specification, Microsoft Research, dec 2013. Released under the Open Specification Promise.
- [SAB⁺19] Alberto Sonnino, Mustafa Al-Bassam, Shehar Bano, Sarah Meiklejohn, and George Danezis. Coconut: Threshold issuance selective disclosure credentials with applications to distributed ledgers. In *ISOC Network and Distributed System Security Symposium – NDSS 2019*, San Diego, CA, USA, February 24–27, 2019. The Internet Society.

- [Sch90] Claus-Peter Schnorr. Efficient identification and signatures for smart cards. In Gilles Brassard, editor, *Advances in Cryptology – CRYPTO’89*, volume 435 of *Lecture Notes in Computer Science*, pages 239–252, Santa Barbara, CA, USA, August 20–24, 1990. Springer, New York, USA.
- [TZ22] Stefano Tessaro and Chenzhi Zhu. Short pairing-free blind signatures with exponential security. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part II*, volume 13276 of *Lecture Notes in Computer Science*, pages 782–811, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.
- [VZK03] Duc Liem Vo, Fangguo Zhang, and Kwangjo Kim. A new threshold blind signature scheme from pairings. In *The 2003 Symposium on Cryptography and Information Security, SCIS 2003, Hamamatsu, Japan, Jan 26-29, 2003 The Institute of Electronics, Information and Communication Engineers*, 2003.